GAI-2109 Lab Guide

Practical Development of Agentic AI Applications

ascendient learning

Part of **Accenture**

Revision 1.0.0 published on 2026-05-20

Ascendient, LLC
1 California Street Suite 2900
San Francisco, CA 94111
https://www.ascendientlearning.com

USA: 1-877-517-6540, email: getinfousa@ascendientlearning.com
Canada: 1-877-812-8887 toll free, email: getinfo@ascendientlearning.com

Lab Materials and Environment Guidelines

Ascendient Learning is pleased to provide lab materials and environments as part of your learning experience. To ensure a safe and effective learning experience for everyone:

You may:

- Use the lab environment for your own coursework.
- Save any important files or data from the environment to your local computer or online storage (if allowed by your employer).
- Retain lab files for your personal use after the class ends.

You may not:

- Share your lab materials or environment with anyone outside of your course.
- Store any personal, private, confidential, or proprietary company information within the lab environment.
- Rely on the lab environment for data storage, as all work is permanently deleted after each class.
- Misuse the lab environment for any illegal, malicious, or non-class-related activities.

> ⓘ Important
>
> The lab environment is under Ascendient Learning's control and is not a part of your organization's IT ecosystem. Do not place your organization's information into the lab environment and always follow your organization's policies and procedures for working with IT systems.

Table of Contents

Building a Basic AI Agent

LAB 1

- ✓ Create a basic AI agent using the Pydantic AI framework
- ✓ Configure agent system prompts to define behavior and personality
- ✓ Execute agents using both synchronous and asynchronous patterns
- ✓ Validate agent responses for correctness and relevance
- ✓ Apply dynamic system prompts with runtime context

1.1. Introduction

AI agents are autonomous programs that use large language models (LLMs) to make decisions, process information, and interact with users. Unlike simple API calls to language models, agents have defined roles, maintained context, and structured interaction patterns.

In this lab, you'll build your first AI agent using Pydantic AI, a Python framework that provides type-safe, structured ways to create and manage AI agents. You'll learn the fundamental concepts that underpin all agent development: initialization, configuration, system prompts, and execution patterns.

1.2. Framework and Tools

Pydantic AI

A Python framework for building production-grade AI agents with:

- Type-safe agent definitions using Pydantic models
- Support for multiple LLM providers (OpenAI, Anthropic, Gemini, etc.)
- Built-in validation and error handling
- Synchronous and asynchronous execution patterns

LLM Provider

This lab works with any LLM provider supported by Pydantic AI. The model is configured via the `AI_MODEL` environment variable in your `.env` file. Choose a model that provides:

- Fast response times suitable for interactive applications
- Good balance of capability and cost

1.3. The Scenario

You're building a customer support system for a software company. The first step is creating a basic support agent that can answer product questions. The agent should:

- Identify itself as a support representative
- Maintain a helpful and professional tone
- Provide accurate information based on its training

This foundational agent will later be enhanced with tools, memory, and integration capabilities.

1.4. Project Structure

Your working directory will contain:

```
building-a-basic-ai-agent/
├── basic_agent.py          # Main agent implementation
├── .env                    # API key configuration (create this)
└── pyproject.toml          # Dependencies (UV format)
```

1.5. Problem Statement

Time Allocation: 45-60 minutes

Objective: Build a basic AI agent using Pydantic AI that acts as a customer support representative, configuring it with a system prompt and executing queries both synchronously and asynchronously.

Expected Outcome: A working Python script that initializes a Pydantic AI agent, runs synchronous and asynchronous queries, inspects execution metadata, and demonstrates dynamic system prompts using dependency injection.

1.6. Getting Started

1. **Set Up Your Environment**

   Navigate to the lab files directory:

   ```
   cd LabFiles/building-a-basic-ai-agent
   ```

2. **Install Dependencies with UV**

   Sync the project dependencies:

   ```
   uv sync
   ```

   This will create a virtual environment and install all required packages defined in `pyproject.toml`.

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**

   Create a `.env` file from the example:

```
cp .env.example .env
```

   Edit `.env` and add your OpenAI API key:

```
OPENAI_API_KEY=your_api_key_here
AI_MODEL=openai:gpt-5.4-mini
```

   Replace `your_api_key_here` with your actual OpenAI API key.

> 💡 Tip
>
> If you don't have an OpenAI API key:
>
> - Visit https://platform.openai.com/api-keys
> - Sign in or create an account
> - Click "Create new secret key"
> - Copy the key and save it securely

> ⚠ Warning
>
> **Troubleshooting API Key Issues:**
>
> If you get an "invalid API key" error:
>
> - Verify the `.env` file is in the **same directory** as `basic_agent.py`
> - Check that there are **no spaces** around the `=` sign: `OPENAI_API_KEY=sk-…`
> - Ensure there are **no quotes** around the API key value
> - Verify the API key starts with `sk-` (for OpenAI keys)
> - Make sure the `.env` file is named exactly `.env` (not `.env.txt`)
>
> You can test if the key is loading by adding this to your script:

```
print(f"API Key loaded: {os.getenv('OPENAI_API_KEY')[:
10]}...")
```

1.7. Core Lab: Build Your First Agent

1. Open the provided `basic_agent.py` file. It contains the starter scaffolding with the agent already initialized:

```
from pydantic_ai import Agent
from dotenv import load_dotenv
import os

# Load environment variables from .env file
load_dotenv()

# Verify API key is loaded
api_key = os.getenv('OPENAI_API_KEY')
if not api_key:
    raise ValueError("OPENAI_API_KEY not found in environment
variables. "
                     "Make sure .env file exists in the same
directory as this script.")

# Get model from environment variable
model = os.getenv('AI_MODEL', 'openai:gpt-5.4-mini')
print(f"Using model: {model}")

# Initialize the agent with a system prompt
agent = Agent(
    model,
    system_prompt="""You are a helpful customer support
representative
    for TechCorp, a software company. You provide clear,
accurate, and
    friendly assistance to customers. Always maintain a
professional
    yet warm tone."""
)

# Main execution
if __name__ == '__main__':
    # YOUR CODE HERE
```


What the Scaffolding Provides:

- Imports Pydantic AI components and environment variable support
- Loads environment variables from the `.env` file
- Validates that the OpenAI API key is present
- Reads the AI model name from the `AI_MODEL` environment variable
- Creates an agent configured with the TechCorp support-rep system prompt

2. Inside the `if __name__ == '__main__':` block, add code that runs a synchronous query against the agent. Your code should:
   - Call `agent.run_sync()` with a test question such as `'What are your support hours?'`
   - Capture the returned object in a variable (for example, `result`)
   - Print the agent's reply using `result.output`
3. Execute your script:

```
uv run basic_agent.py
```

Validation Checkpoint: You should see output similar to:

```
Using model: openai:gpt-5.4-mini
Agent Response: Our customer support team is available Monday
through
Friday, 9:00 AM to 6:00 PM EST. For urgent issues outside these
hours,
you can submit a ticket through our support portal, and we'll
respond
within 24 hours.
```

> ⓘ Note
>
> The exact response will vary because LLM outputs are non-deterministic. However, the response should be relevant to the question and maintain the professional tone specified in the system prompt.

4. Extend your `main` block to test the agent across multiple scenarios. Your code should:
   - Define a list of at least three different support questions (for example: password reset, product features, login troubleshooting)
   - Iterate over the list and call `agent.run_sync()` for each question
   - Print both the query and the agent's response on each iteration so you can compare them

   Run the script again and observe how the agent maintains its persona across different questions.

   Validation Checkpoint: Each response should maintain the professional, helpful tone defined in the system prompt.

5. Add code to inspect the execution metadata for a query. Your code should:
   - Call `agent.run_sync()` with a question such as `'What payment methods do you accept?'` and store the result
   - Print the agent's response with `result.output`
   - Print the total tokens used, available via `result.usage().total_tokens` (guard against `result.usage()` returning `None`)

   Validation Checkpoint: You should see the response followed by execution metadata showing the total number of tokens used.

1.8. Challenge 1: Implement Dynamic System Prompts

Open the provided `challenge1.py` file. It contains the starter scaffolding: a `Department` Pydantic model, an agent configured with `deps_type=Department`, and a `@agent.system_prompt` decorated function that builds the prompt from `ctx.deps.name` and `ctx.deps.expertise` each time the agent runs.

> ⓘ Note
>
> Pydantic AI does **not** f-string-interpolate values into a static `system_prompt=` argument — literal `{department.name}` text in a static

> string would be sent to the LLM verbatim and produce a plausible-looking but wrong reply. Use the `@agent.system_prompt` decorator (as the starter does) whenever your prompt needs to read from `deps`.

Goal: Complete the *`main`* block so the same agent can represent different departments (Sales, Support, Billing) based on runtime parameters.

Hint: Pass a `Department` instance through the `deps=` argument of `agent.run_sync()`. Pydantic AI binds it to `ctx.deps` and re-runs the decorated `department_system_prompt` function on every call, so each department gets a system prompt built from its own values.

Your Task:

- Create at least three `Department` instances with distinct `name` and `expertise` values (for example: Customer Support, Sales, Billing)
- Define a single query you want to send to every department (for example, `"Can you help me?"`)
- Call `agent.run_sync()` once per department, passing the query and the matching `Department` as `deps=`
- Print each response, labeled with the department name, so you can compare how the agent's behavior shifts

Validation: The agent's responses should reflect the department context provided.

1.9. Challenge 2: Implement Asynchronous Execution

Open the provided `challenge2.py` file. It contains the starter scaffolding: `asyncio` is imported, the agent is initialized with the support-rep system prompt, and an empty `async def main()` function is waiting for you to fill in.

Goal: Replace synchronous agent calls with asynchronous calls and execute multiple queries in parallel.

Hint: Use Pydantic AI's async `agent.run()` method (instead of `run_sync()`) together with `asyncio.gather()` for concurrent execution.

Your Task inside `async def main()`:

- Define a list of at least three different queries
- Build a list of coroutines by calling `agent.run(query)` for each query (do not `await` them individually)
- Use `await asyncio.gather(*tasks)` to run them concurrently and collect the results
- Iterate over the queries and results together and print each pair so you can verify the output

Your Task in *`main`*:

- Invoke the async `main` coroutine with `asyncio.run()`

Validation: All three queries should execute simultaneously and complete faster than sequential execution.

1.10. Next Steps

Explore Further:

- Experiment with different LLM models by changing the `AI_MODEL` environment variable (e.g., `anthropic:claude-sonnet-4-5` or `google:gemini-2.5-flash`)
- Create agents with different personalities and expertise areas
- Measure and compare response times between synchronous and asynchronous execution
- Add error handling for API failures and rate limits

1.11. Review

In this lab you learned:

- How to initialize a Pydantic AI agent with a system prompt that defines its role and personality
- How to execute agent queries using both synchronous and asynchronous patterns
- How to inspect execution metadata such as token usage
- How to use dependency injection with `deps_type` and `RunContext` for dynamic system prompts
- How to run multiple agent queries in parallel using `asyncio.gather()`

Enhancing a Basic Agent with Prompts and Configurations

LAB 2

- ✓ Design effective system prompts using structured prompt engineering techniques
- ✓ Configure model parameters (temperature, max_tokens) to control output behavior
- ✓ Compare different model tiers and their performance characteristics
- ✓ Implement dynamic context injection for runtime prompt customization
- ✓ Enable streaming responses for improved user experience

2.1. Introduction

While a basic agent can respond to queries, production agents require careful tuning of both their prompts and model configurations. System prompts define the agent's behavior, personality, and output format, while model parameters control creativity, verbosity, and determinism.

In this lab, you'll transform a basic agent into a production-ready assistant by:

- Crafting detailed system prompts with clear instructions
- Experimenting with model parameters to optimize behavior
- Comparing different model capabilities
- Implementing advanced features like streaming

2.2. Framework and Tools

Pydantic AI Model Configuration

Pydantic AI provides fine-grained control over LLM behavior through:

- `temperature` - Controls randomness (0.0 = deterministic, 2.0 = very creative)
- `max_tokens` - Limits response length
- `top_p` - Nucleus sampling for diversity
- Model selection across providers

Model Tiers

You'll compare three model tiers, configured via environment variables so the lab works against any provider that exposes a fast / mid / capable lineup:

- `AI_MODEL_FAST` (default `openai:gpt-5.4-nano`) — fast and economical for simple tasks

- `AI_MODEL` (default `openai:gpt-5.4-mini`) — strong performance, cost-effective; this is the agent's default model
- `AI_MODEL_CAPABLE` (default `openai:gpt-5.4`) — most capable, for complex tasks

2.3. The Scenario

You're building an AI writing assistant for a content marketing team. The assistant needs to:

- Generate blog post outlines with specific formatting
- Maintain a consistent brand voice (professional but approachable)
- Adapt its verbosity based on the task (concise summaries vs. detailed articles)
- Provide real-time feedback as it generates content

This requires precise prompt engineering and careful model configuration to ensure consistent, high-quality outputs.

2.4. Project Structure

```
enhancing-agent/
├── enhanced_agent.py       # Main agent with configurations
├── .env                    # API credentials
└── pyproject.toml          # Dependencies (UV format)
```

2.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Transform a basic agent into a production-ready AI writing assistant by crafting detailed system prompts, configuring model parameters, and enabling streaming responses.

Expected Outcome: A working agent that generates structured blog outlines with consistent brand voice, demonstrates the effects of different temperature settings, and streams responses token-by-token to the terminal.

2.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/enhancing-a-basic-agent-with-prompts-and-configurations
   ```
2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```
   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.
3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```
   Edit `.env` and add your OpenAI API key:
   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```
4. **Review the Starter File**
   Open the provided `enhanced_agent.py`. The scaffolding you'll use throughout the Core Lab is already in place:
   - API-key loading and validation
   - A detailed system prompt constant named `DETAILED_SYSTEM_PROMPT` (brand voice, output format, quality standards)

- An `agent` initialized with that prompt and `ModelSettings(temperature=0.7, max_tokens=1000)`
- A helper function `test_temperature(temperature, query)` that creates a temporary agent at the given temperature and prints the result
- A helper function `compare_models(query)` that runs the same query against three model tiers — fast / default / capable — read from the `AI_MODEL_FAST`, `AI_MODEL`, and `AI_MODEL_CAPABLE` environment variables (with the OpenAI 5.4 family as defaults)
- An empty `if __name__ == '__main__':` block containing `# YOUR CODE HERE`

  Read through each piece so you understand what's available to call from `__main__`.

2.7. Core Lab: Exercise the Enhanced Agent

All of your work in the Core Lab happens inside the `if __name__ == '__main__':` block of `enhanced_agent.py`. Complete the three tasks below in sequence, running the script after each one to verify behavior.

1. **Run the Enhanced Agent**

   In `__main__`, add code that:

   - Calls `agent.run_sync()` with a query such as `"Write a blog outline about AI agents"`
   - Prints the agent's response using `result.output`
   - Prints the total tokens used via `result.usage().total_tokens` (guard against `result.usage()` returning `None`)

   Run the script:

   ```
   uv run enhanced_agent.py
   ```

   Validation Checkpoint: The output should follow the format specified by `DETAILED_SYSTEM_PROMPT` — a clear title, structured sections, and a

call-to-action — constrained to approximately 1000 tokens. You'll see the token count displayed after the output.

2. **Compare Temperature Settings**
   Extend your `main` block to call `test_temperature()` three times with the same query, using temperatures `0.0` (deterministic), `0.7` (balanced), and `1.5` (more creative). Reuse the same query each time so the outputs are directly comparable.

   Run the script again.

   Validation Checkpoint: Higher temperatures should produce more varied and creative outputs, while lower temperatures are more consistent and focused.

3. **Compare Model Tiers**
   Further extend your `main` block to call `compare_models()` with a blog-outline query.

   Run the script again.

   Validation Checkpoint: More capable models should produce more nuanced and detailed outlines compared to smaller models, but at higher token cost.

2.8. Challenge 1: Dynamic Context Injection (Optional)

This challenge is optional — attempt it if you finish the Core Lab early or explore it after class.

Open `challenge1.py` (the starter). It provides a `ContentContext` Pydantic model (`audience`, `length`, `tone`) and a `build_dynamic_prompt(ctx)` helper, with an empty `main` block. A reference implementation lives at `solutions/challenge1_solution.py` once you're ready to compare your work.

Your Task: In `main`, create three `ContentContext` instances with different combinations, and for each one build a fresh `Agent` using `build_dynamic_prompt(context)` as the `system_prompt`. Run the same query (e.g. `"Create a blog outline about microservices architecture"`) against each and print a labeled header plus `result.output`.

Validation: Technical audience should include more jargon; casual tone should be more conversational; shorter lengths should produce fewer sections.

2.9. Challenge 2: Implement Streaming Output (Optional)

This challenge is optional — attempt it if time permits.

Open `challenge2.py` (the starter). It provides imports, API-key validation, the `DETAILED_SYSTEM_PROMPT`, a configured `agent`, and empty `stream_response()`, `main()`, and `main` blocks. A reference implementation lives at `solutions/challenge2_solution.py` once you're ready to compare your work.

Your Task:

- Inside `stream_response()`, use `async with agent.run_stream(query) as response:` and iterate `async for chunk in response.stream_text(delta=True):`, printing each chunk with `end=''` and `flush=True`. After the stream, print `response.usage().total_tokens` (guard against `None`).
- Inside `main()`, await `stream_response()` for two or three different blog-outline queries.
- In `main`, invoke `main` via `asyncio.run()`.

Validation: Output should appear progressively, token by token, rather than all at once.

2.10. Next Steps

Explore Further:

- Experiment with other model parameters (`top_p`, `frequency_penalty`, `presence_penalty`)
- Create a library of reusable prompt templates for different content types
- Implement A/B testing to compare prompt variations
- Add prompt validation to catch common mistakes before execution

2.11. Review

In this lab you learned:

- How to craft structured system prompts with brand voice guidelines, output format rules, and quality standards
- How to configure model parameters like temperature and max_tokens to control output creativity and length
- How to compare different model tiers to evaluate capability and cost trade-offs
- How to inject dynamic context at runtime to customize agent behavior for different audiences and tones
- How to enable streaming responses for real-time token-by-token content generation

Enforcing Structured Outputs with Pydantic Models

LAB 3

- ✓ Define Pydantic models to enforce structured output schemas for AI agents
- ✓ Configure agents with result types for automatic validation
- ✓ Implement field validators and constraints for data quality
- ✓ Handle validation failures gracefully with retry mechanisms
- ✓ Use Union types to support multiple output formats

3.1. Introduction

Free-form text responses from LLMs are flexible but difficult to process programmatically. Production applications need structured, validated data that downstream systems can consume reliably. Pydantic AI solves this by allowing you to define strict output schemas using Pydantic models.

When you specify an `output_type`, Pydantic AI:

1. Instructs the LLM to return data matching your schema
2. Parses the LLM's response into your Pydantic model
3. Validates all fields and constraints
4. Automatically retries on validation failures

This lab teaches you to transform unpredictable LLM outputs into reliable, type-safe data structures.

3.2. Framework and Tools

Pydantic Models

Python's most popular data validation library, providing:

- Type-safe data classes with automatic validation
- Field constraints (min/max values, regex patterns, custom validators)
- Nested models for complex data structures
- JSON serialization and deserialization

Pydantic AI Result Types

Integrates Pydantic models with LLM agents:

- `output_type` parameter defines expected output schema
- Automatic validation of LLM responses
- Built-in retry logic for validation failures

- Support for complex types (Union, Optional, Lists)

3.3. The Scenario

You're building a product review analysis system for an e-commerce platform. The system needs to:

- Extract structured information from customer reviews (rating, sentiment, key points)
- Ensure all extracted data meets quality standards (ratings 1-5, valid sentiments)
- Handle reviews in multiple languages with consistent output format
- Feed structured data into analytics dashboards and recommendation engines

Free-form text won't work—you need validated, structured data that your systems can trust.

3.4. Project Structure

```
files/
├── .env.example            # Copy to .env and add your API key
├── pyproject.toml          # Dependencies (UV format)
├── models.py               # Pydantic schemas (provided — read-
only)
├── review_analyzer.py      # Core Lab — complete the __main__ block
├── retry_analyzer.py       # Challenge 1 — complete
analyze_with_retry() and __main__
└── union_analyzer.py       # Challenge 2 — complete the __main__
block
```

3.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Exercise a product review analysis agent that enforces structured Pydantic output schemas, validates extracted data fields, and handles validation failures with retry mechanisms.

Expected Outcome: A working agent that parses customer reviews into validated `DetailedReviewAnalysis` objects with constrained fields, a retry wrapper that re-prompts with clearer instructions when validation fails, and a Union-typed agent that returns different structured formats for product versus service feedback.

3.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/enforcing-structured-outputs-with-pydantic-models
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

4. **Review the Starter Files**
   Your lab directory contains several starter files. Open and read each one so you know what's already available:
   - `models.py` — Pydantic schemas for review analysis (`ReviewAnalysis`, `Pros`, `Cons`, `DetailedReviewAnalysis`, `ProductReviewAnalysis`, `ServiceAnalysis`). All six classes are defined for you.
   - `review_analyzer.py` — Core Lab scaffolding: imports, API-key validation, model selection, and an `agent` already configured with `output_type=DetailedReviewAnalysis`. The `if __name__ == '__main__':` block is marked `# YOUR CODE HERE`.
   - `retry_analyzer.py` — Challenge 1 scaffolding: imports (including `ValidationError` and `asyncio`), an `agent` configured with `output_type=ReviewAnalysis`, and an `async def analyze_with_retry(review, max_retries=3)` stub whose body is marked `# YOUR CODE HERE`.
   - `union_analyzer.py` — Challenge 2 scaffolding: imports, an `agent` configured with `output_type=Union[ProductReviewAnalysis, ServiceAnalysis]`, and an empty `__main__` block.

3.7. Core Lab: Exercise the Structured Output Agent

All of your Core Lab work happens inside the `if __name__ == '__main__':` block of `review_analyzer.py`. The schemas and the agent are already provided; your job is to run the agent against a sample review and print the validated, nested output. The key schema is `DetailedReviewAnalysis` in `models.py` — it contains a `rating` (1–5), `sentiment`, `summary`, nested `pros` and `cons` lists, and a `recommended` boolean. The `agent` in `review_analyzer.py` is already configured with `output_type=DetailedReviewAnalysis`; just call it with a review string. Add the following steps inside the `if __name__ == '__main__':` block.

1. **Define a sample review and run the agent**
   Create a `review` string containing a multi-sentence customer review that mixes positive and negative points so the agent has material to populate

both `pros` and `cons`. Then call `agent.run_sync()` with a prompt that asks the agent to analyze the `review`, and capture the returned object in a variable such as `result`.

2. **Access the structured result**
   Read the parsed output through `result.output` — it will already be a `DetailedReviewAnalysis` instance, so no manual parsing is required.

   Hint: `result.output.pros` is a `Pros` instance, not a plain list — iterate over `result.output.pros.items` to reach the strings.

3. **Print the validated fields**
   Print each of the following on its own labeled line: `rating` (formatted as `X/5`), `sentiment`, `summary`, every entry in `pros.items` (one per line), every entry in `cons.items` (one per line), and `recommended`.
4. **Run the script and verify the output**
   Execute the analyzer from the terminal:

```
uv run review_analyzer.py
```

   You should see structured output resembling:

```
Using model: openai:gpt-5.4-mini
Rating: 4/5
Sentiment: positive
Summary: <brief summary under 200 characters>

Pros:
  + <positive aspect 1>
  + <positive aspect 2>
  ...

Cons:
  - <negative aspect 1>
  ...

Recommended: True
```

If any field violates a constraint (for example, a rating outside 1–5 or a sentiment outside the four allowed values), Pydantic AI will automatically re-prompt the model with corrective feedback before surfacing an error.

3.8. Challenge 1: Implement Custom Retry Logic (Optional)

Open the provided `retry_analyzer.py` file. It contains the starter scaffolding:

- Imports including `asyncio` and `ValidationError`
- API-key validation and model selection
- An `agent` configured with `output_type=ReviewAnalysis` and a review-analyst system prompt
- An `async def analyze_with_retry(review: str, max_retries: int = 3)` stub whose body is marked `# YOUR CODE HERE`
- An empty `if __name__ == '__main__':` block marked `# YOUR CODE HERE`

Goal: Implement the async retry wrapper so it reruns the agent with progressively clearer instructions when Pydantic validation fails, and surfaces the validated `ReviewAnalysis` once one succeeds.

Your Task inside `analyze_with_retry()`:

- Loop up to `max_retries` times, tracking the current attempt number
- On the first attempt, build a simple prompt that just asks the agent to analyze the review text
- On later attempts, build a more explicit prompt that re-states the schema constraints in plain language (the integer rating range, the allowed sentiment vocabulary, the summary length cap, and the required key-points count) so the model has another chance to comply
- Call the async `agent.run(prompt)` (not `run_sync`) and, on success, return `result.output`
- Catch `ValidationError`, print the failure so the student can see what went wrong, and fall through to the next attempt

- If the final attempt also fails, re-raise the `ValidationError` rather than returning `None`

Your Task in `main`:

- Define a short review string to feed the wrapper
- Invoke the async wrapper via `asyncio.run()` and capture the returned `ReviewAnalysis`
- Print a few fields from the analysis (for example, rating, sentiment, summary, and the list of key points) so success is visible

Run the retry example:

```
uv run retry_analyzer.py
```

Validation: The function should return a validated `ReviewAnalysis` on success. If validation fails on an attempt, you should see a printed error followed by another attempt; after all retries are exhausted, the `ValidationError` should propagate.

Extension: Log each attempt's raw output before validation so you can see exactly what the model returned and why it failed.

3.9. Challenge 2: Support Multiple Output Formats with Union Types (Optional)

Open the provided `union_analyzer.py` file. It contains the starter scaffolding:

- Imports including `Union` from `typing` and both `ProductReviewAnalysis` and `ServiceAnalysis` from `models`
- API-key validation and model selection
- An `agent` configured with `output_type=Union[ProductReviewAnalysis, ServiceAnalysis]` and a system prompt instructing the model to pick the right schema based on whether the review is about a product or a service
- An empty `if __name__ == '__main__':` block marked `# YOUR CODE HERE`

The `ProductReviewAnalysis` and `ServiceAnalysis` classes (already in `models.py`) share `rating` and `sentiment` but otherwise diverge — product reviews include `key_features` and `recommended`, service reviews include `service_quality`, `staff_rating`, and `would_return`.

Goal: Feed the same agent one product review and one service review, confirm it selects the correct output model for each, and print the type-specific fields.

Your Task in `main`:

- Define two short review strings: one clearly about a **product** and one clearly about a **service**
- Call `agent.run_sync()` on each review and capture the two results
- Print the concrete class name of each result (use `type(result.output).name`) so you can see which branch of the Union was chosen
- Print the shared fields (`rating`, `sentiment`) for both results
- Use `isinstance(result.output, ServiceAnalysis)` to guard access to the service-only fields, and print `staff_rating` and `would_return` when that guard passes

Run the union type example:

```
uv run union_analyzer.py
```

Validation: The product review should return a `ProductReviewAnalysis` and the service review should return a `ServiceAnalysis`. The type-name printout should show the two different class names, and the service-only fields should print only for the service review.

Extension: Add a third analysis type for app reviews with metrics like `ui_rating`, `performance_rating`, and `feature_completeness`, widen the agent's `output_type` Union to include it, and feed an app review through the same pipeline.

3.10. Next Steps

Explore Further:

- Implement logging to track validation failures and retry patterns
- Create a library of reusable model schemas for common tasks
- Compare validation success rates across different models by changing the `AI_MODEL` environment variable
- Add confidence scores to indicate model certainty

3.11. Review

In this lab you learned:

- How to define Pydantic models with field constraints and custom validators to enforce output schemas
- How to configure agents with `output_type` for automatic response validation and parsing
- How to build nested Pydantic models for complex structured outputs like pros/cons breakdowns
- How to implement retry logic that provides progressively clearer instructions on validation failures
- How to use Union types to support multiple output formats from a single agent

Data Extraction Agent with Validated Output

LAB 4

- ✓ Design prompts that extract structured data from unstructured text
- ✓ Define Pydantic schemas with optional fields for incomplete data
- ✓ Validate extracted data for completeness and accuracy
- ✓ Handle missing or ambiguous information gracefully
- ✓ Implement confidence scoring for extracted fields

4.1. Introduction

Unstructured text—emails, documents, invoices, support tickets—contains valuable information locked in narrative form. Data extraction agents transform this unstructured text into structured, validated data that systems can process.

Unlike simple parsing, AI-powered extraction handles:

- Natural language variations ("$500", "five hundred dollars", "500 USD")
- Missing or implicit information
- Context-dependent interpretation
- Multiple formats and layouts

In this lab, you'll build an agent that extracts structured business data from unstructured sources with validation to ensure reliability.

4.2. Framework and Tools

Pydantic AI Extraction Pattern

Combines prompt engineering with structured outputs:

- Prompt guides the model to find specific information
- Pydantic schema defines expected structure
- Validation ensures data quality
- Optional fields handle incomplete data

Field Validation Strategies

- Required vs. Optional fields
- Default values for missing data
- Custom validators for domain-specific rules
- Confidence scores to flag uncertain extractions

4.3. The Scenario

You're building an automated invoice processing system for the accounting department. The system receives invoices in various formats (emails, PDFs converted to text, scanned documents) and needs to:

- Extract key information (vendor, amount, date, items)
- Validate extracted data (valid dates, reasonable amounts)
- Flag incomplete or uncertain extractions for human review
- Feed clean data to the accounting system

Manual data entry is error-prone and time-consuming. Your extraction agent will automate 90% of invoices, flagging only problematic cases for human attention.

4.4. Project Structure

```
files/
├── .env.example            # Copy to .env and add your API key
├── pyproject.toml          # Dependencies (UV format)
├── models.py               # Pydantic schemas (provided — read-
only)
├── sample_invoices.py      # Test invoice data (provided — read-
only)
├── invoice_extractor.py    # Core Lab — complete the __main__ block
├── challenge1_solution.py  # Challenge 1 — complete the __main__
block
└── challenge2_solution.py  # Challenge 2 — complete
robust_extract() and __main__
```

4.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build an automated invoice processing agent that extracts structured data from unstructured invoice text in multiple formats and validates the results using Pydantic schemas with confidence scoring.

Expected Outcome: A working extraction pipeline that processes three sample invoices of varying formats, returns validated `InvoiceData` objects with optional fields handled gracefully, and flags low-confidence extractions for human review.

4.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/data-extraction-agent-with-validated-output
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

4. **Review the Starter Files**
   Your lab directory contains several starter files. Open and read each one so you know what's already available:
   - `models.py` — Pydantic schemas for invoice data (`LineItem`, `InvoiceData`, `InvoiceDataWithConfidence`, `PartialInvoiceData`). All four classes are defined for you.
   - `sample_invoices.py` — Five test invoice strings (`INVOICE_1`, `INVOICE_2`, `INVOICE_3`, `NOISY_INVOICE`, `PROBLEMATIC_INVOICE`).
   - `invoice_extractor.py` — Core Lab scaffolding: an agent configured with a confidence-scoring system prompt, an `extract_invoice()` helper, and an empty `main` block marked `# YOUR CODE HERE`.
   - `challenge1.py` — Challenge 1 scaffolding: an agent configured to handle noisy input, an `extract_invoice()` helper, and an empty `main`.
   - `challenge2.py` — Challenge 2 scaffolding: a `main_agent`, a `robust_extract()` function whose body is marked `# YOUR CODE HERE`, and an empty `main`.

4.7. Core Lab: Exercise the Extraction Agent

All of your Core Lab work happens inside the `if __name__ == '__main__':` block of `invoice_extractor.py`. The schemas and the agent are already provided; your job is to exercise them against the sample invoices. The Core Lab agent uses the `InvoiceDataWithConfidence` schema from `models.py` — it extends `InvoiceData` with `vendor_confidence` and `amount_confidence` floats (0.0–1.0) and a `needs_review()` method that flags extractions with either confidence below 0.7 or a missing invoice number. Use the provided `extract_invoice(invoice_text)` helper rather than calling the agent directly.

1. **Build the Invoice List**
   In the `if __name__ == '__main__':` block, build a list of `(label, invoice_text)` pairs covering `INVOICE_1`, `INVOICE_2`, and `INVOICE_3`. Use short, human-readable labels (for example, "ABC Office Supplies", "ACME Catering", "Informal Email").

2. **Loop and Print Headers**
   Iterate the list. For each pair, print a header line with the label so the output sections are clearly separated.
3. **Call `extract_invoice()` Safely**
   Inside the loop, call `extract_invoice(invoice_text)` inside a `try`/`except` block so a single bad extraction doesn't abort the whole run. On failure, print the exception.
4. **Print the Extracted Fields**
   On success, print each of the following on its own line:
   - `vendor_name` together with `vendor_confidence` (formatted to two decimal places)
   - `total_amount` together with `amount_confidence` (format the amount as currency)
   - `invoice_number` (fall back to `'N/A'` when missing)
   - `invoice_date` (fall back to `'N/A'` when missing)
   - The boolean result of `extracted.needs_review()`
5. **Run and Verify**
   Run the extractor:

   ```
   uv run invoice_extractor.py
   ```

   Validation Checkpoint:

   - All three invoices extract without raising an exception.
   - `INVOICE_1` (ABC Office Supplies) and `INVOICE_2` (ACME Catering) extract with high confidence scores and report `Needs Review: False`.
   - `INVOICE_3` (the informal email) has lower confidence scores — vendor is inferred and there is no invoice number — and reports `Needs Review: True`.

4.8. Challenge 1: Handle Input Variations

This challenge is optional — complete it only if you have time remaining after the Core Lab.

Open the provided `challenge1.py` file. It contains the starter scaffolding:

- Imports (including `NOISY_INVOICE` from `sample_invoices`)
- API-key validation and model selection
- An `agent` already configured with a system prompt that instructs the model to handle OCR errors (0→O, 1→I), typos, inconsistent formatting, and ambiguous dates, while still returning confidence scores
- An `extract_invoice()` helper wrapping `agent.run_sync()`
- An empty `if __name__ == '__main__':` block marked `# YOUR CODE HERE`

The `sample_invoices.py` file defines `NOISY_INVOICE`, which contains OCR-style substitutions such as `"1NV01CE N0"`, `"Vend0r: Qu1ckSupply C0rp"`, and an informal due date (`"March 25th 2024"`).

Goal: Run the noise-tolerant agent against `NOISY_INVOICE` and confirm it still produces a clean, validated `InvoiceDataWithConfidence`.

Your Task in `__main__`:

- Print a header identifying the test run
- Wrap the extraction in `try`/`except` so failures don't crash the run
- Call `extract_invoice(NOISY_INVOICE)` and, on success, print vendor name, total, invoice number, and date — each alongside its confidence score where one is available — plus the result of `needs_review()`
- On failure, print the exception

Validation: The agent should normalize `"Qu1ckSupply C0rp"` to something like `"QuickSupply Corp"`, parse `$3,150.00` as the total, and convert the due date into ISO format (`YYYY-MM-DD`). Confidence scores should remain reasonable given the noisy source.

4.9. Challenge 2: Implement Fallback Strategies

This challenge is optional — complete it only if you have time remaining after the Core Lab.

Open the provided `challenge2.py` file. It contains the starter scaffolding:

- Imports (including `INVOICE_1` and `PROBLEMATIC_INVOICE` from `sample_invoices`, plus `InvoiceDataWithConfidence` and `PartialInvoiceData` from `models`)
- API-key validation and model selection
- A `main_agent` configured with `output_type=InvoiceDataWithConfidence` and a full-extraction system prompt
- A `robust_extract(invoice_text)` function whose signature is complete (`→ InvoiceDataWithConfidence | PartialInvoiceData`) but whose body is marked `# YOUR CODE HERE`
- An empty `if __name__ == '__main__':` block marked `# YOUR CODE HERE`

The `PartialInvoiceData` schema (already in `models.py`) has optional `vendor_name` and `total_amount` fields, a required `raw_text_snippet` string, and an `extraction_errors` list.

Goal: When the full extraction fails, fall back to a partial extraction that captures whatever can be read and records the problems encountered — no exception should propagate out of `robust_extract()`.

Hint: Build a second `Agent` locally inside the `except` branch with `output_type=PartialInvoiceData` and a short system prompt telling the model to extract what it can and populate `extraction_errors`.

Your Task inside `robust_extract()`:

- Try running `main_agent.run_sync()` against the invoice text and return its `.output`
- On any exception, log that the full extraction failed (include the error message), then build a fallback `Agent` with `output_type=PartialInvoiceData`, run it against the same text, and return its `.output`

Your Task in `main`:

- Call `robust_extract(INVOICE_1)` first; you should get an `InvoiceDataWithConfidence` back. Confirm with `isinstance()` and print a couple of fields from the result
- Then call `robust_extract(PROBLEMATIC_INVOICE)`; you should get a `PartialInvoiceData` back. Confirm with `isinstance()` and print whatever fields are available, including `extraction_errors` and a short `raw_text_snippet` preview

Validation: The good invoice should return a fully validated `InvoiceDataWithConfidence`. The problematic invoice should degrade gracefully to `PartialInvoiceData` with populated `extraction_errors` — no exception should propagate out of `robust_extract()`.

4.10. Next Steps

Explore Further:

- Build extraction agents for other document types (receipts, contracts, resumes)
- Implement batch processing for multiple documents
- Add post-processing rules for business-specific logic
- Create a review queue UI for low-confidence extractions

4.11. Review

In this lab you learned:

- How to design extraction schemas with required and optional fields for handling incomplete data
- How to write system prompts that guide precise information extraction from unstructured text

- How to process documents in varied formats including formal invoices and informal emails
- How to add confidence scoring to flag uncertain extractions for human review
- How to implement fallback strategies that degrade gracefully when full extraction fails

Integrating a Custom Tool into an AI Agent

LAB 5

- ✓ Create custom tool functions with clear input/output contracts
- ✓ Register tools with Pydantic AI agents using the `@agent.tool` decorator
- ✓ Design effective tool descriptions that guide agent decision-making
- ✓ Implement error handling in tools for robust agent behavior
- ✓ Integrate multiple tools and enable intelligent tool selection

5.1. Introduction

Out of the box, AI agents can only generate text. To interact with the real world—look up data, perform calculations, call APIs, query databases—agents need tools.

Tools are Python functions that:

- The agent can invoke when needed
- Have clear input parameters (typed and documented)
- Return results to the agent
- Handle errors gracefully

Pydantic AI makes tool integration seamless with decorators that automatically generate tool descriptions and schemas that LLMs can understand.

In this lab, you'll transform a basic agent into a practical assistant by integrating custom tools for real-world tasks.

5.2. Framework and Tools

Pydantic AI Tool System

Features for tool integration:

- `@agent.tool` decorator for registration
- Automatic function signature analysis
- Type annotations become tool schemas
- Docstrings become tool descriptions
- Support for sync and async tools

Tool Design Principles

- Single responsibility—one tool, one task
- Clear, descriptive names and parameters
- Robust error handling

- Type-safe inputs and outputs

5.3. The Scenario

You're building a customer service agent for an e-commerce company. The agent needs to:

- Look up customer order status
- Calculate shipping costs
- Check product availability
- Process refund requests

Without tools, the agent can only make things up. With tools, it can provide accurate, real-time information by querying actual systems.

5.4. Project Structure

```
files/
├── customer_agent.py       # Core Lab — complete the tool bodies
and __main__ block
├── challenge1.py           # Challenge 1 — complete
complete_order_inquiry() and __main__
├── challenge2.py           # Challenge 2 — complete ToolAnalytics,
tool bodies, and __main__
├── mock_database.py        # Simulated orders and inventory data
(provided — read-only)
├── .env.example            # Copy to .env and add your API key
└── pyproject.toml          # Dependencies (UV format)
```

5.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build a customer service agent that integrates custom tools for order status lookup, inventory checking, and shipping calculation, enabling the agent to provide accurate real-time information.

Expected Outcome: A working agent with three registered tools that correctly selects the appropriate tool for each query, handles error conditions gracefully, and includes instrumentation for tracking tool usage analytics.

5.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/integrating-a-custom-tool-into-an-ai-agent
   ```
2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```
   This will create a virtual environment and install all required packages defined in `pyproject.toml`.
3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```
   Edit `.env` and add your OpenAI API key:
   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```
4. **Review the Starter Files**
   The following files are provided:
   - `customer_agent.py` — Imports, `load_dotenv()`, agent initialization, and three tool functions (`get_order_status`, `check_inventory`,

`calculate_shipping`) decorated with `@agent.tool_plain` and fully documented with type hints and docstrings. The `check_inventory` and `calculate_shipping` tool bodies are pre-built as reference examples. The `get_order_status` body and the *`main`* block are marked `# YOUR CODE HERE`.

- `challenge1.py` — Imports, agent initialization, and the three Core Lab tools already implemented. A fourth tool, `complete_order_inquiry`, is declared with its signature and docstring but the body is marked `# YOUR CODE HERE`. The *`main`* block is also empty.
- `challenge2.py` — Imports (including `defaultdict` and `datetime`), a `ToolAnalytics` class skeleton with *`init`*, `record_call`, and `get_stats` method stubs, an `analytics` instance, agent initialization, and three tool stubs whose bodies and the *`main`* block are marked `# YOUR CODE HERE`.
- `mock_database.py` — Provided data module exporting `ORDERS` and `INVENTORY` dictionaries. No changes needed.
- `.env.example` — Template for environment variables.

5.7. Core Lab: Implement a Custom Tool

Open the provided `customer_agent.py` file. Two tools (`check_inventory` and `calculate_shipping`) are already implemented as reference examples so you can see how tool bodies are structured. Your job is to implement `get_order_status` using the same pattern and then wire up the *`main`* block to exercise all three tools.

1. **Review the Pre-Built Tool Examples**
   Skim `check_inventory` and `calculate_shipping` in `customer_agent.py`. Notice the shared pattern: guard on a missing key with a clear error message, then read from the mock data source and return a formatted string. You will follow this shape in the next step.

2. **Implement `get_order_status`**
   Inside the body of the `get_order_status` function, replace `# YOUR CODE HERE` with logic that:
   - Checks whether `order_id` exists as a key in the `ORDERS` dictionary; if not, return a clear "not found" message that includes the missing `order_id`
   - Otherwise, reads the order record and pulls out its status, total, and tracking value
   - Returns a single formatted string that includes the order id, status, total (formatted as currency), and tracking value

     Hint: Access `ORDERS[order_id]` only after the key check. Use an f-string for the return value. Mirror the shape of the pre-built `check_inventory` tool. For `tracking`, note that some orders have the key present with a value of `None` (look at `ORD-002`), so `dict.get("tracking", "Not yet available")` returns `None`, **not** the default. Guard with an `or` fallback (`order.get("tracking") or "Not yet available"`) instead of relying on `.get()'s second argument.

3. **Exercise the Tools from `main`**
   Inside the `if __name__ == "__main__":` block, add code that:
   - Runs at least three queries through `agent.run_sync()`, one that should trigger each tool (an order-status question, an inventory question, and a shipping question)
   - Prints a short label before each call (so you can see which tool fired) and prints `result.output` after each call
4. **Run and Validate**
   Run the agent:

   ```
   uv run customer_agent.py
   ```

   Validation Checkpoint: The agent should automatically select the right tool for each query, return natural-language responses that include the tool's data (including order status, stock levels, and shipping quotes), and reject invalid shipping inputs cleanly. No Python exceptions should bubble up.

5.8. Challenge: Reimplement the Reference Tools From Scratch

The `check_inventory` and `calculate_shipping` tools were provided to you pre-built in the Core Lab. To deepen your understanding of the tool body pattern, delete both implementations in your `customer_agent.py` and rewrite them from scratch without referring to the originals.

Goal: Reproduce each tool's behavior based only on its docstring and the requirements below.

Your Task inside `check_inventory`:

- Checks whether `product_name` exists as a key in `INVENTORY`; if not, return a "not found" message naming the product
- Reads the product record and extracts the available count and price
- Returns different strings depending on whether available units are greater than zero (in-stock message with count and price) or zero (out-of-stock message with price)

Your Task inside `calculate_shipping`:

- Validates `weight_kg` — return an error message if the weight is not positive, and another error message if it exceeds a reasonable maximum (pick a sensible upper bound such as 30 kg)
- Looks up a base rate for the destination from a small lookup table covering at least US, CA, UK, and MX, with a fallback rate for unsupported destinations
- Applies a weight-based surcharge (for example, a small per-kg charge for weight above 1 kg)
- Looks up an estimated delivery window for the destination, with a fallback for unsupported destinations
- Returns a single string that includes the destination (normalized to uppercase), the computed cost (formatted as currency), and the delivery estimate

Hint: Use `.get(key, default)` for clean fallbacks in the shipping lookup tables.

Validation: Queries like "Is Keyboard in stock?" should communicate availability and price; shipping queries should produce realistic quotes for supported countries and reject invalid weights with a helpful message.

5.9. Challenge 1: Multi-Tool Workflow

Open the provided `challenge1.py` file. It contains the starter scaffolding: all Core Lab imports, the agent, the three completed tools from the Core Lab (`get_order_status`, `check_inventory`, `calculate_shipping`), and a new tool `complete_order_inquiry(order_id: str) → str` whose body is marked `# YOUR CODE HERE`. The `main` block is empty.

Goal: Build a "complete order inquiry" tool that combines data from multiple sources to produce a single comprehensive response about an order, including current inventory status for every item in that order.

Hint: You do not need to call the other tools to do this. Read directly from `ORDERS` and `INVENTORY` and build up a multi-line string.

Your Task inside `complete_order_inquiry()`:

- If `order_id` is missing from `ORDERS`, return a "not found" message
- Otherwise, start a response string with the order header (id, status, total)
- Iterate over the order's `items` list; for each item, look it up in `INVENTORY` and append a line showing whether it is in stock and its price (handle the case where an item is missing from inventory)
- Append a tracking line, using a sensible placeholder when `tracking` is `None`
- Return the assembled string

Your Task in `main`:

- Call `agent.run_sync()` with at least one query that asks for complete details on a known order
- Print the agent's response

Run the agent:

```
uv run challenge1.py
```

Validation: The agent should invoke `complete_order_inquiry` and produce a single response that weaves order status together with per-item availability.

Extension: Add a tool that requires calling other tools internally (for example, a "prepare reorder" tool that checks inventory and calculates shipping).

5.10. Challenge 2: Implement Tool Usage Analytics

Open the provided `challenge2.py` file. It contains the starter scaffolding: imports (including `defaultdict` from `collections` and `datetime` from `datetime`), a `ToolAnalytics` class with `__init__`, `record_call`, and `get_stats` method stubs each marked `# YOUR CODE HERE`, an `analytics = ToolAnalytics()` instance, an `Agent` initialization, and three tool stubs (`get_order_status`, `check_inventory`, `calculate_shipping`) whose bodies are marked `# YOUR CODE HERE`. The `__main__` block is empty.

Goal: Add instrumentation that records every tool invocation so you can see which tools are used most and whether each call succeeded or failed.

Your Task inside `ToolAnalytics.__init__`:

- Initialize a container that counts calls per tool name (a `defaultdict(int)` works well)
- Initialize a list that will hold per-call history records

Your Task inside `ToolAnalytics.record_call(tool_name, args, success)`:

- Increment the counter for `tool_name`
- Append a history record that captures at least the current timestamp (`datetime.now()`), the tool name, the arguments dict, and the success flag

Your Task inside `ToolAnalytics.get_stats()`:

- Compute the total number of recorded calls
- Print a header and a per-tool summary showing call count (and optionally a percentage) sorted by most-used first
- Print the last few history records with their timestamp, tool name, and arguments, and a visible success/failure marker

Your Task inside each tool (`get_order_status`, `check_inventory`, `calculate_shipping`):

- Re-implement the same logic you wrote in the Core Lab
- Call `analytics.record_call()` before every return point, passing the tool name, the relevant arguments as a dict, and a success flag that reflects whether the call produced a useful answer (missing keys, invalid weights, and exceptions count as failures)
- Wrap the work in `try` / `except` so that unexpected exceptions still get recorded before re-raising

Your Task in `main`:

- Build a list of at least a half-dozen queries that exercise all three tools, with some repeats so counts are interesting
- Run each query through `agent.run_sync()`
- After the loop, call `analytics.get_stats()` to display the results

Run the agent:

```
uv run challenge2.py
```

Validation: After the queries complete, you should see a summary showing which tools were invoked most often and a short history of recent calls with success indicators.

5.11. Next Steps

Explore Further:

- Integrate tools that call actual APIs (weather, currency conversion, etc.)
- Build tools that query real databases
- Implement rate limiting for expensive tool operations
- Create tool categories for better organization

5.12. Review

In this lab you learned:

- How to create tool functions with clear input/output contracts using type hints and docstrings
- How to register tools with a Pydantic AI agent using the `@agent.tool` decorator
- How to implement error handling within tools so the agent communicates failures gracefully
- How to enable an agent to intelligently select between multiple registered tools based on user queries
- How to add instrumentation to track tool usage analytics for optimization and debugging

Tool Design Patterns for Common Integrations

LAB 6

- ✓ Design read-only lookup tools with structured output schemas
- ✓ Implement search tools with filtering and pagination
- ✓ Model errors explicitly for robust failure handling
- ✓ Add observability features (logging, correlation IDs) to tools
- ✓ Implement write operations with idempotency patterns

6.1. Introduction

Production agents need tools that integrate with real systems: REST APIs, databases, file systems, and external services. Ad-hoc tool implementations quickly become brittle and difficult to maintain.

This lab teaches proven design patterns for building robust integration tools:

- **Query/Lookup Pattern** - Retrieve single records by ID
- **Search Pattern** - Find records matching criteria
- **Command Pattern** - Perform write operations safely
- **Error Modeling** - Explicit error types for different failures
- **Observability** - Tracing, logging, and debugging support

These patterns make tools reliable, maintainable, and production-ready.

6.2. Framework and Tools

httpx Library

Modern HTTP client for Python:

- Async/sync support
- Timeout configuration
- Connection pooling
- Retry strategies

Pydantic for Tool Schemas

Use Pydantic models to define:

- Tool input parameters
- Tool output structures
- Error responses

Tool Design Principles

- Explicit error handling over exceptions
- Structured responses (success/error distinction)
- Idempotent operations where possible
- Comprehensive logging

6.3. The Scenario

You're building an AI customer service agent that needs to integrate with multiple backend systems:

- **User Service API** - Customer account information
- **Order Service API** - Order history and status
- **Product Catalog API** - Product details and availability
- **Knowledge Base** - Support articles and FAQs

Each integration requires robust error handling, retries, timeouts, and observability. You'll build reusable tool patterns that demonstrate best practices for production use.

6.4. Project Structure

```
files/
├── .env.example            # Copy to .env and add your API key
├── pyproject.toml          # Dependencies (UV format)
├── mock_api.py             # Mock backend data and functions
(provided — read-only)
├── tools/
│   ├── __init__.py         # Package marker (provided)
│   ├── models.py           # Pydantic schemas (provided — read-
only)
│   └── api_tools.py        # Core Lab — complete lookup_user();
search_user_orders() is pre-built as a reference example
├── agent.py                # Agent that wires up the tools
(provided — read-only)
├── challenge1.py           # Challenge 1 — complete update_user()
and __main__
└── challenge2.py           # Challenge 2 — complete correlation
helpers, tool wrappers, and __main__
```

6.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build a production-ready query/lookup integration tool for a customer service agent using the `ToolResponse` envelope pattern with explicit error types and logging.

Expected Outcome: A working agent with a structured-response lookup tool backed by a mock API, demonstrating graceful error handling and best practices for production tool integration.

6.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:

   ```
   cd LabFiles/tool-design-patterns-for-common-integrations
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:

   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:

   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

4. **Review the Starter Files**
   Your lab directory contains several starter files. Open and read each one so you know what's already available:
   - `mock_api.py` — Simulated backend. Defines the `USERS`, `ORDERS`, and `PRODUCTS` dictionaries and exposes `get_user(user_id)` and `search_orders(user_id, status)` helper functions.
   - `tools/models.py` — Pydantic schemas: `ToolResponse` (standard success/error envelope with `success`, `data`, `error_type`, `error_message`), `User`, and `Order`.
   - `tools/api_tools.py` — Contains `lookup_user(user_id)` with the body marked `# YOUR CODE HERE` for you to implement. Also contains a

fully pre-built `search_user_orders(user_id, status, limit)` function you can read as a reference example of the Search tool pattern.

- `agent.py` — A complete Pydantic AI agent wired to both tool functions, with test prompts in *`main`*. Read it to see how the tools are registered.
- `challenge1.py` — Challenge 1 scaffolding: imports, an `UpdateRequest` Pydantic model, a `processed_requests` cache dict, an agent, and a decorated `update_user()` whose body and *`main`* are marked `# YOUR CODE HERE`.
- `challenge2.py` — Challenge 2 scaffolding: imports (including `contextvars` and `uuid`), a `correlation_id_var` ContextVar, empty `get_correlation_id()` and `log_with_correlation()` helpers, an agent, and decorated `lookup_user()` / `search_user_orders()` tool wrappers — all bodies and *`main`* are marked `# YOUR CODE HERE`.

6.7. Core Lab: Implement the Query/Lookup Tool Pattern

All Core Lab work happens inside `tools/api_tools.py`. The schemas (`tools/models.py`), the mock backend (`mock_api.py`), the agent wiring (`agent.py`), and the search tool (`search_user_orders`, a pre-built reference example) are already in place. Your job is to implement `lookup_user(user_id)` so the agent can call it.

Key API shapes to know: `get_user(user_id)` returns `{"success": True, "data": {…}}` on success or `{"success": False, "error": "…", "message": "…"}` on failure. Your tool must return a `ToolResponse` envelope (with `success`, `data`, `error_type`, and `error_message`) serialized to JSON.

1. **Read the pre-built search tool as a reference**
   Before you write any code, open `tools/api_tools.py` and read `search_user_orders()`. It demonstrates the same `ToolResponse` envelope, logging, and `try`/`except` pattern you'll apply to `lookup_user()`.
2. **Add an entry log line**
   Inside `lookup_user()`, emit an informational log line (via `logger.info`) announcing the lookup — include the `user_id` in the message.

3. **Wrap the body in `try` / `except`**
   Start a `try` block for the main logic and an `except Exception as e:` branch. In the `except` branch, log at `error` level and return a `ToolResponse` with `success=False`, `error_type="SystemError"`, and `error_message=str(e)` — serialized to JSON.
4. **Call the backend and handle the success branch**
   Inside the `try` block, call `get_user(user_id)`. If `result["success"]` is true, validate `result["data"]` through the `User` Pydantic model, then build a `ToolResponse` with `success=True` and `data=user.model_dump()`. Log an info-level success message.
5. **Handle the not-found branch**
   In the `else` branch (when `result["success"]` is false), build a `ToolResponse` with `success=False`, `error_type=result["error"]`, and `error_message=result["message"]`. Log a warning-level message noting the user was not found.
6. **Return JSON on every path**
   Return `response.model_dump_json()` so every code path yields a JSON string — never a Pydantic object.

   Hint: Every `return` statement must return a JSON string. The `ToolResponse.model_dump_json()` method handles serialization for you.

7. **Run and validate**
   Run the agent:

   ```
   uv run agent.py
   ```

   Validation Checkpoint:

   - The agent successfully looks up known users and reports errors clearly for unknown users.
   - The log output shows an info line on entry, and either an info (success) or warning (not-found) line after the backend call.
   - Calling `lookup_user("user-001")` in a Python REPL returns a JSON string whose parsed form has `success: true` and a populated

`data.name`. Calling `lookup_user("user-999")` returns `success: false` with `error_type: "UserNotFound"`.

Pattern Elements to Notice:

- Standardized response envelope (`ToolResponse`) used by both `lookup_user` and the pre-built `search_user_orders`
- Explicit error types (`UserNotFound`, `SystemError`)
- Structured input/output via Pydantic models for type safety
- Comprehensive logging at entry, success, and failure points

6.8. Challenge: Re-implement the Search Tool Pattern from Scratch

The Search tool pattern has been pre-built in `tools/api_tools.py` as `search_user_orders()` so you could focus on the lookup pattern. As a challenge, re-implement it yourself without looking at the provided version.

Goal: Recreate the search behavior under a new function name so it returns the same `ToolResponse` envelope structure as the provided version.

Setup: In `tools/api_tools.py`, **add** a new function named `search_user_orders_v2(user_id, status, limit)` **alongside** the existing `search_user_orders` (do not rename or comment out the original — `challenge2.py` imports the original `search_user_orders` symbol, so removing it breaks Challenge 2's imports). Then point `agent.py's `@agent.tool_plain` -wrapped `search_user_orders` at your new `search_user_orders_v2` implementation while you exercise it; revert that delegation when you finish the challenge so the rest of the lab keeps using the reference implementation.

Your implementation should:

- Log the search parameters for observability
- Call `search_orders()` from the mock API, forwarding the filter arguments

- Trim the returned records to at most `limit` entries, and validate each one through the `Order` model
- On success, build a `ToolResponse` whose `data` is a dict containing the list of orders (serialized), the trimmed count, and the untrimmed total from the backend response
- On a backend failure, return a `ToolResponse` with `success=False` and `error_type="SearchError"`
- Wrap the body in a `try` / `except` that degrades any unexpected exception to `error_type="SystemError"` with the stringified message
- Return the `ToolResponse` as JSON on every path

Hint: Use a list comprehension to build the validated `Order` list, and a second one to serialize them for the response.

Validation:

- Running `uv run agent.py` with "Show me all shipped orders" causes the agent to call `search_user_orders` with a `status` filter and returns at least one order.
- "What orders does user-001 have?" causes the agent to call `search_user_orders` with a `user_id` filter.
- The log output shows an info line for each tool call and a count of orders found.

6.9. Challenge: Implement Write Operation Tool

Open the provided `challenge1.py` file. It contains the starter scaffolding:

- Imports, `load_dotenv()`, logging configuration
- The `UpdateRequest` Pydantic model (fields: `user_id`, optional `email`, optional `tier`, required `idempotency_key`)
- A module-level `processed_requests = {}` dict for caching completed updates

- An `agent` configured with a system prompt that instructs the model to always supply a unique idempotency key
- A decorated `update_user(user_id, email, tier, idempotency_key)` tool whose body is marked `# YOUR CODE HERE`
- An empty `if __name__ == '__main__':` block marked `# YOUR CODE HERE`

Goal: Complete `update_user()` so that repeating the same call with the same idempotency key returns the cached result instead of mutating the user record a second time.

Hint: Check the cache before doing any work, and only write to the cache after a successful update.

Your Task inside `update_user()`:

- If an `idempotency_key` was supplied and is already a key in `processed_requests`, log the duplicate detection and return the cached response directly
- Otherwise, validate that the user exists by calling `get_user()`; if it doesn't, return a `ToolResponse` with `success=False` and `error_type="UserNotFound"` as JSON
- Mutate `USERS[user_id]` in place, applying only the fields the caller supplied (`email` and/or `tier`), skipping values that already match the current record
- Build a success `ToolResponse` whose `data` contains the updated user dict, a record of which fields changed, and an ISO-formatted timestamp (`datetime.now().isoformat()`)
- Serialize the response to JSON, store it in `processed_requests` under the `idempotency_key` (if one was supplied), and return it
- Wrap everything in a `try`/`except` that returns a JSON `ToolResponse` with `error_type="UpdateError"` on unexpected failure

Your Task in `main`:

- Run three agent queries through `agent.run_sync()`:
  - A first update to some user's email, supplying a specific idempotency key
  - A duplicate request that uses the **same** idempotency key but asks for a **different** email — the agent output should still reflect the first email because the cached response is returned
  - A third update to a different user's tier, supplying a new idempotency key
- Print clear separators and labels between the three test cases so the idempotency behavior is obvious in the output

Validation: Calling the same update twice with the same idempotency key must return the cached result without modifying the stored user record. The third call with a fresh key should actually apply its change.

6.10. Challenge: Implement Correlation ID Tracking

Open the provided `challenge2.py` file. It contains the starter scaffolding:

- Imports including `contextvars`, `uuid`, and the existing `lookup_user` / `search_user_orders` implementations from `tools.api_tools` (aliased as `lookup_user_impl` / `search_orders_impl`)
- Logging configured with a timestamped format string
- A module-level `correlation_id_var = contextvars.ContextVar('correlation_id', default=None)`
- Empty `get_correlation_id()` and `log_with_correlation()` helpers marked `# YOUR CODE HERE`
- An `agent` configured for this challenge
- Decorated `lookup_user()` and `search_user_orders()` tool wrappers whose bodies are marked `# YOUR CODE HERE`
- An empty `if __name__ == '__main__':` block

Goal: Give every user request a single correlation ID so that all log lines emitted during that request (including those from the tool calls) can be filtered by that ID.

Hint: `ContextVar` values propagate across function calls within the same execution context, which is exactly what you need to share a correlation ID between the agent call and the tools it invokes.

Your Task inside `get_correlation_id()`:

- Read the current value from `correlation_id_var`
- If it's unset, generate a new UUID string, store it on the ContextVar, and return it; otherwise return what was already there

Your Task inside `log_with_correlation()`:

- Fetch the correlation ID via `get_correlation_id()`
- Use `logger.log()` with the level resolved dynamically from the string argument (e.g., via `getattr(logging, level)`), and include the correlation ID as a prefix in the log message (format: `[correlation_id=…] <message>`)

Your Task inside the decorated tool wrappers:

- `lookup_user(user_id)` — log a correlation-aware "starting" message, delegate to `lookup_user_impl(user_id)`, log a "completed" message, and return the impl's result. On exception, log at ERROR level and re-raise
- `search_user_orders(user_id, status)` — same pattern, delegating to `search_orders_impl(user_id, status)`

Your Task in *`main`*:

- For each of several distinct agent requests (at minimum: a single-tool lookup, a multi-tool question that forces both tools to fire, and a search-only query), explicitly set `correlation_id_var` to a fresh `uuid.uuid4()` string before calling `agent.run_sync()`
- Log a "request started" / "request completed" line through `log_with_correlation()` around each call so the boundary is visible
- Print the agent's output for each request with clear separators

Validation: When you run the script, every log line emitted during a single request — including the tool-level entries — shares the same `correlation_id=…` prefix. Different requests produce different IDs, making it trivial to grep for the lifecycle of any one request.

6.11. Next Steps

Explore Further:

- Add HTTP timeout and retry handling with libraries like `httpx` and `tenacity` when integrating with real APIs
- Implement circuit breaker pattern for failing services
- Add caching layer to reduce API calls
- Build connection pooling for database tools
- Create monitoring dashboards for tool performance

6.12. Review

In this lab you learned:

- How to design lookup tools with standardized response formats and explicit error types
- How to implement search tools with filtering and pagination for querying backend data
- How to build write operations with idempotency patterns to prevent duplicate side effects
- How to implement correlation ID tracking for end-to-end request tracing across tool calls

Managing Dependencies in Agent Applications

LAB 7

- ✓ Implement dependency injection for AI agents using RunContext
- ✓ Design clean architectures separating business logic from agent logic
- ✓ Manage database connections and external resources properly
- ✓ Ensure state isolation between concurrent agent instances
- ✓ Build testable agents with injected dependencies

7.1. Introduction

Production agents need access to external resources—databases, APIs, caches, configuration. Hardcoding these creates tight coupling, makes testing difficult, and causes resource leaks.

Dependency injection solves these problems:

- **Testability** - Mock dependencies in tests
- **Flexibility** - Swap implementations easily
- **Resource management** - Proper lifecycle control
- **State isolation** - Independent agent instances
- **Clean architecture** - Separation of concerns

Pydantic AI provides `RunContext` for dependency injection, enabling production-grade agent architectures.

7.2. Framework and Tools

Dependency Injection (DI)

- Pass dependencies to components rather than hardcoding
- Enables testing, flexibility, and proper resource management

RunContext

- Pydantic AI's DI mechanism
- Provides dependencies to tools and system prompts
- Type-safe dependency passing via `deps_type`

State Isolation

- Each agent run gets independent context
- No shared mutable state between runs
- Thread-safe concurrent execution

7.3. The Scenario

You're building an order management agent that needs:

- **Database access** - Query orders, customers, inventory
- **Cache layer** - Reduce database load
- **Email service** - Send notifications
- **Configuration** - API keys, timeouts, feature flags
- **Logger** - Structured logging

These resources must be injected, not hardcoded, for testability and proper lifecycle management.

7.4. Project Structure

```
files/
├── .env.example                          # Copy to .env and add your
API key
├── pyproject.toml                        # Dependencies (UV format)
├── dependencies.py                       # Dependency dataclasses
(provided — read-only)
├── database.py                           # Database pool management
(provided — read-only)
├── factory.py                            # Dependency factory
(provided — read-only)
├── agent.py                              # Core Lab — complete
get_order(), process_order_query(), and main()
├── test_agent.py                         # Test suite (provided —
run to verify)
├── challenge1_scoped_dependencies.py     # Challenge 1 — complete
resolve(), begin_request(), dispose(), example_usage()
└── challenge2_ioc_container.py           # Challenge 2 — complete
resolve(), _create_instance(), get_order(), example_usage(),
test_circular_dependency()
```

7.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build an order management agent that uses Pydantic AI's `RunContext` to inject a database, cache, email service, and configuration, demonstrating proper resource management and state isolation between concurrent runs.

Expected Outcome: A working agent with tools that transparently access injected dependencies, a dependency factory that manages resource lifecycle, and a pytest suite that passes using mocked dependencies to verify cache-hit, cache-miss, and isolation behavior.

7.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/managing-dependencies-in-agent-applications
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key. The file provides reasonable defaults for the other variables (model name, database URL, cache host/port, email settings, environment flags).

4. **Review the Starter Files**

   Your lab directory contains several starter files. Open and read each one so you know what is already available:

   - `dependencies.py` — Five dataclasses that model the injected resources: `DatabaseConnection`, `CacheClient`, `EmailService`, `AppConfig`, and a container `AgentDependencies`. Each class provides the async methods your agent tools will call (for example `database.query()`, `cache.get()` / `cache.set()`, `email.send()`, `config.get_feature()`).
   - `database.py` — A `DatabasePool` class that manages a pool of `DatabaseConnection` instances with `initialize()`, `shutdown()`, and an async `get_connection()` context manager.
   - `factory.py` — A `DependencyFactory` that reads environment variables, creates the database pool, cache client, email service, and config, and wraps them in an `AgentDependencies` container. Also exposes `shutdown()` for cleanup.
   - `agent.py` — Core Lab scaffolding: an `OrderInfo` Pydantic model, an `order_agent` already constructed with `deps_type=AgentDependencies` and a system prompt, a reference `check_feature_enabled()` tool, two fully implemented reference tools (`send_order_update()`, `search_orders()`) you can read to see the DI pattern, one tool stub (`get_order()`) marked `# YOUR CODE HERE`, a `process_order_query()` helper stub, and an empty `main()`.
   - `test_agent.py` — A complete pytest suite that uses `MockDatabase`, `MockCache`, and `MockEmail` to test the tools you will write. Run it to verify your implementation.
   - `challenge1_scoped_dependencies.py` — Challenge 1 scaffolding: a `DependencyScope` enum, a `DependencyRegistration` dataclass, and a `ScopedDependencyContainer` with `register()`, `_create_instance()`, `end_request()`, and `_dispose_instance()` already implemented. Methods marked `# YOUR CODE HERE` are for you to complete.
   - `challenge2_ioc_container.py` — Challenge 2 scaffolding: a `Lifetime` enum, `ServiceDescriptor` dataclass, `CircularDependencyError`, and

an `IoCContainer` with `register_singleton()`, `register_transient()`, `register_instance()`, `dispose()`, and `_dispose_instance()` already implemented. Sample interfaces (`ILogger`, `IDatabase`, `ICache`) and implementations are provided.

7.7. Core Lab: Implement the Order Agent Tools

All of your Core Lab work happens inside `agent.py`. The agent, the `OrderInfo` output model, the system prompt, and the tool signatures are already provided. Each tool takes `ctx: RunContext[AgentDependencies]` as its first parameter and reaches injected resources through `ctx.deps.database`, `ctx.deps.cache`, `ctx.deps.email`, and `ctx.deps.config`. A complete `check_feature_enabled()` tool is included as a reference example, and two additional reference tools (`send_order_update()` and `search_orders()`) are fully implemented so you can see the DI pattern applied to different resources.

1. **Review the reference tools**
   Open `agent.py` and read the three completed tools (`check_feature_enabled`, `send_order_update`, `search_orders`). Note how each one accepts `ctx: RunContext[AgentDependencies]` and accesses `ctx.deps.config`, `ctx.deps.email`, and `ctx.deps.database`. You will follow the same pattern for the tool you implement next.
2. **Implement `get_order()`**
   Fill in the body of the `get_order` tool so it implements a **cache-aside** lookup pattern.

   Your Task:

   - Build a cache key from the incoming `order_id` (use a namespace prefix such as `order:` so different keys don't collide)
   - Check the cache first via `ctx.deps.cache.get()`; if the value is present, convert it back into an `OrderInfo` and return it immediately
   - On cache miss, call `ctx.deps.database.query()` with a SQL statement that selects by id and a params dict containing the `order_id`

- If the database returns an empty list, raise `ValueError` with a message that mentions the missing `order_id`
- Otherwise, construct an `OrderInfo` from the first row (fields: `id`, `status`, `customer_email`, `total`, `created_at`)
- Populate the cache with `order.model_dump()` before returning, so the next call is a cache hit

Validation Checkpoint: After completing this task, the `test_get_order_cache_miss` and `test_get_order_cache_hit` tests in `test_agent.py` should pass.

3. **Implement `process_order_query()`**
   Inside `process_order_query()`, call `order_agent.run()` with the `query` and the `deps` argument, then return its `.output`.
4. **Implement `main()`**
   Your Task inside `main()`:
   - Instantiate `DependencyFactory()` (already imported for you)
   - Inside a `try`/`finally`, call `factory.create_dependencies()` to get an `AgentDependencies` instance
   - Send one or two sample queries through `process_order_query()` to exercise the tools (for example, an order status question). Print each query and the response
   - In the `finally` branch, call `factory.shutdown()` so the database pool is closed

   Validation Checkpoint: Run the agent end-to-end:

   ```
   uv run agent.py
   ```

   You should see initialization logs, the agent calling tools to answer each query, and clean shutdown of the database pool.

5. **Run the test suite**
   With the three tasks complete, run the provided test suite:

   ```
   uv run pytest test_agent.py -v
   ```

Validation Checkpoint: All tests should pass — cache hit, cache miss, email send, feature-disabled, feature-flag check, dependency isolation, concurrent requests, order-not-found, database-error, and cache operations.

7.8. Challenge 1: Implement Scoped Dependencies

Open the provided `challenge1_scoped_dependencies.py` file. It contains the starter scaffolding: a `DependencyScope` enum with three values (`SINGLETON`, `REQUEST`, `TRANSIENT`), a generic `DependencyRegistration` dataclass, and a `ScopedDependencyContainer` with several methods already implemented (`register`, `_create_instance`, `end_request`, `_dispose_instance`). The methods you must complete are marked `# YOUR CODE HERE`. Sample service classes (`DatabaseService`, `RequestContext`, `Logger`) are provided to exercise each scope.

Goal: Complete the container so that resolving a dependency returns the right instance for its scope — one shared instance for singletons, one per active request for request scope, and a fresh instance every call for transient scope.

Hints:

- Singleton — cache the first created instance on the registration and reuse it forever
- Request — cache the instance on the container under the request's lifetime and clear that cache when a new request begins
- Transient — never cache; create a new instance on every call
- `_create_instance()` already handles both sync and async factories for you

Your Task inside `resolve()`:

- Raise `ValueError` if the name has not been registered
- Look up the registration and branch on its scope, handing off to `_create_instance()` when a new instance is needed
- Return the singleton, request-scoped, or transient instance as appropriate

Your Task inside `begin_request()`:

- Increment the internal request counter
- Reset the per-request instance cache so a fresh set of request-scoped dependencies is created next time

Your Task inside `dispose()`:

- Iterate over every registration and, for any singleton that has been instantiated, call `_dispose_instance()`
- Do the same for any request-scoped instances still cached
- Finally, clear the registrations and the request cache

Your Task inside `example_usage()`:

- Instantiate the container and register `DatabaseService` as `SINGLETON`, `RequestContext` as `REQUEST`, and `Logger` as `TRANSIENT`
- Begin a request, resolve each dependency twice, and print whether the two results share the same `instance_id` — singleton and request should match, transient should not
- End the request, begin a second one, and resolve again to show that the singleton persists across requests while request-scoped instances do not
- Call `await container.dispose()` at the end for cleanup

Approach: See presentation module for dependency scoping patterns.

Validation: Running `uv run challenge1_scoped_dependencies.py` should print comparisons showing singleton equality across both requests, request-scope equality within a request but not across, and a transient instance count equal to the number of `resolve()` calls for `Logger`.

7.9. Challenge 2: Build an IoC Container

Open the provided `challenge2_ioc_container.py` file. It contains the starter scaffolding: a `Lifetime` enum, a `ServiceDescriptor` dataclass, a `CircularDependencyError` exception, and an `IoCContainer` with

`register_singleton()`, `register_transient()`, `register_instance()`, `dispose()`, and `_dispose_instance()` already implemented. Sample interfaces (`ILogger`, `IDatabase`, `ICache`) and implementations (`ConsoleLogger`, `SqlDatabase`, `MemoryCache`, `OrderService`) are provided so you can demonstrate automatic resolution.

Goal: Register services and resolve them automatically using constructor type hints, with proper lifetime handling and circular-dependency detection.

Hints:

- Keep a set of "currently resolving" types on the container so you can detect cycles
- For singletons, cache the instance on the descriptor after the first build
- Use `inspect.signature()` and a parameter's `annotation` to learn what types a constructor needs
- `dispose()` is already implemented and will call `dispose()` or `close()` on your singletons for you

Your Task inside `resolve()`:

- Raise `ValueError` if the service type is not registered
- If the descriptor is a singleton with a cached instance, return it directly
- If the service type is already in the "resolving" set, raise `CircularDependencyError` with a message that shows the chain
- Otherwise, mark the type as resolving, delegate to `_create_instance()`, cache the result if the lifetime is singleton, and remove the marker in a `finally` block

Your Task inside `_create_instance()`:

- If the implementation is a callable but not a class (a factory function), invoke it, await if it returned a coroutine, and return the result
- Otherwise, introspect the constructor signature, skip `self`, and for each parameter that has a type annotation, recursively `await self.resolve(annotation)` and collect the results as keyword arguments

- Instantiate the class with those kwargs and return the instance

Your Task inside `OrderService.get_order()`:

- Try the cache first, returning any hit immediately
- On miss, call `self.database.query()` with a SQL string that references the `order_id`, store the result in the cache under `order_id`, and return it

Your Task inside `example_usage()`:

- Instantiate the container and register `ILogger`, `IDatabase`, and `ICache` as singletons (with their respective concrete implementations) and `OrderService` as transient
- Resolve `OrderService` twice and print whether the two `logger`, `database`, and service instances are the same — singletons should match, the service itself should not
- Call `await container.dispose()` at the end

Your Task inside `test_circular_dependency()`:

- Create a container, register two transient classes whose constructors reference each other, and call `resolve()` on one of them
- The call must raise `CircularDependencyError`; catch it and print a confirmation

Approach: See presentation module for IoC container implementation.

Validation: Running `uv run challenge2_ioc_container.py` should print the resolution output followed by a confirmation that the circular-dependency case was detected.

7.10. Challenge 3: Re-implement the Reference Tools from Scratch

The Core Lab provides `send_order_update()` and `search_orders()` as fully implemented reference tools so you can see the DI pattern applied to different

resources. For additional practice, delete the bodies of those two tools and re-implement them yourself without peeking.

Your Task for `send_order_update()`:

- Call `get_order()` (reusing the tool you implemented) to retrieve the order — this also warms the cache
- Check the `email_notifications` feature flag via `ctx.deps.config.get_feature()`; if disabled, return `False` without sending
- Otherwise call `ctx.deps.email.send()` with the recipient email from the order, a subject that includes the `order_id`, and the `message` argument as the body
- Return the boolean result of the send call

Your Task for `search_orders()`:

- Call `ctx.deps.database.query()` with a SQL statement that selects orders by customer email and a params dict containing the `customer_email` argument
- Convert each row in the result into an `OrderInfo` (same fields as `get_order`) and return the list

Validation: After completing this challenge, `test_send_order_update` and `test_send_order_update_feature_disabled` should still pass, and the agent should be able to answer queries that require either tool.

7.11. Next Steps

Explore Further:

- Implement advanced dependency scoping
- Build IoC container with auto-resolution
- Add health checks for dependencies
- Implement dependency monitoring

7.12. Review

In this lab you learned:

- How to use Pydantic AI's `RunContext` for type-safe dependency injection into agent tools
- How to design clean architectures that separate business logic from agent logic
- How to manage database connections, caches, and external services as injectable dependencies
- How to ensure state isolation between concurrent agent runs using independent dependency instances
- How to write tests with mocked dependencies to verify agent behavior without external services

Dependency Lifecycle Management and Configuration

LAB 8

- ✓ Manage dependency lifecycles with initialization and cleanup hooks
- ✓ Implement configuration management with type-safe settings
- ✓ Handle resource cleanup gracefully even during failures
- ✓ Build context managers for automatic resource management
- ✓ Implement request-scoped and singleton dependency patterns

8.1. Introduction

Production agent applications must manage resources carefully:

- **Database connections** - Initialize pools, close cleanly
- **HTTP clients** - Maintain connection pools, handle timeouts
- **Cache connections** - Open connections, flush on shutdown
- **Configuration** - Load from environment, validate on startup
- **Secrets** - Secure API keys, rotate credentials
- **Cleanup** - Release resources even on failures

Without proper lifecycle management:

- Resource leaks accumulate over time
- Connections remain open unnecessarily
- Configuration errors surface late
- Testing becomes difficult
- Deployments fail unpredictably

This lab teaches production patterns for managing dependencies throughout their complete lifecycle.

8.2. Lifecycle Stages

Initialization

```
Load config → Validate → Initialize connections → Ready
```

Runtime

```
Request → Get/Create resources → Execute → Release → Repeat
```

Shutdown

```
Flush caches → Close connections → Save state → Exit
```

8.3. The Scenario

You're building a production agent service that uses:

- **PostgreSQL** - Customer and order data (connection pool)
- **Redis** - Response caching (persistent connection)
- **HTTP client** - External API calls (connection pooling)
- **S3 bucket** - Document storage (client with credentials)
- **Configuration** - Environment-based settings

All resources must:

- Initialize cleanly on startup
- Be reused efficiently during runtime
- Clean up properly on shutdown
- Handle failures gracefully

8.4. Project Structure

```
files/
├── .env.example                  # Copy to .env and add your API
key
├── pyproject.toml                # Dependencies (UV format)
├── config.py                     # Type-safe Settings (provided —
read-only)
├── dependencies.py               # Database, cache, HTTP clients
(provided — read-only)
├── lifecycle_manager.py          # Core Lab — complete initialize()
and shutdown()
├── agent.py                      # Provided — runs the agent
against managed dependencies
├── test_lifecycle.py             # Tests with fake dependencies
(provided — read-only)
├── challenge1_health_checks.py   # Challenge — complete
check_database(), check_cache(), check_all(), get_health_report()
└── challenge2_graceful_shutdown.py # Challenge — complete
shutdown(), _wait_for_requests(), handle_request(), and
example_graceful_shutdown()
```

8.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build a production agent service with full dependency lifecycle management, using Pydantic Settings for type-safe configuration, context managers for automatic resource cleanup, and a LifecycleManager that initializes and shuts down all dependencies in correct dependency order.

Expected Outcome: A working service that initializes PostgreSQL, Redis, and HTTP client connections on startup, uses them in agent tools, cleans them up reliably on shutdown even during failures, and integrates cleanly with FastAPI's lifespan hook and a pytest suite using fake dependencies.

8.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/dependency-lifecycle-management-and-configuration
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

4. **Review the Starter Files**
   Your lab directory contains several starter files. Open and read each one so you know what's already available:
   - `config.py` — A complete `Settings` class (Pydantic Settings v2) with validated fields for application, OpenAI, database, Redis, HTTP, feature flags, and security. Includes `load_config()`, `configure_logging()`, `is_production()`, `get_masked_config()`, and `validate_production_config()`. Read-only.
   - `dependencies.py` — Three fully-implemented client classes: `DatabaseClient`, `CacheClient`, and `HTTPClient`. Each exposes `initialize()`, `close()`, operation methods, and a `get_stats()` helper. Read-only.

- `lifecycle_manager.py` — Contains a `ManagedDependencies` dataclass and a `LifecycleManager` class whose `initialize()` and `shutdown()` methods are marked `# YOUR CODE HERE`. A private `_cleanup_dependencies()` helper, the `lifespan()` context manager, and a complete `health_check()` are already provided.
- `agent.py` — A complete Pydantic AI agent wired to `ManagedDependencies` with cache-aside `lookup_order` and `get_external_data` tools plus a working `process_query()` and `main()`. You will run it end-to-end to verify the lifecycle plumbs through to a real agent.
- `test_lifecycle.py` — A complete pytest suite with `FakeDatabaseClient`, `FakeCacheClient`, and `FakeHTTPClient` fixtures plus coverage for initialization, context manager, error cleanup, health checks, and double initialization. Read-only — run it to validate your Core Lab work.
- `challenge1_health_checks.py` — A `HealthChecker` class with `check_database()`, `check_cache()`, `check_all()`, and `get_health_report()` marked `# YOUR CODE HERE`. `check_http_client()` and `get_overall_status()` are already implemented as references.
- `challenge2_graceful_shutdown.py` — A `ShutdownConfig` dataclass and a `GracefulShutdownManager` class whose `shutdown()`, `_wait_for_requests()`, `handle_request()` methods, plus the `example_graceful_shutdown()` demo, are marked `# YOUR CODE HERE`. Request-counter bookkeeping and the FastAPI example are provided.

8.7. Core Lab: Implement Lifecycle Management

You will complete the two most important methods of `LifecycleManager` — the orchestration layer that starts up and tears down every dependency in the right

order. `config.py`, `dependencies.py`, `agent.py`, and `health_check()` are already provided.

1. **Complete `LifecycleManager.initialize()`**
   In `lifecycle_manager.py`, replace the `# YOUR CODE HERE` marker inside `initialize()`:
   - Return `self.dependencies` early if `self._initialized` is already `True`
   - Declare `database = cache = http = None` before the `try:` block so the cleanup helper can see whatever got built
   - Construct `DatabaseClient(self.settings)`, `CacheClient(self.settings)`, and `HTTPClient(self.settings)`, then await each one's `initialize()` in order (database, cache, HTTP)
   - Wrap the three clients in `ManagedDependencies(…)`, assign to `self.dependencies`, set `self._initialized = True`, and return the container
   - In `except Exception as e:`, log with `exc_info=True`, call `await self._cleanup_dependencies(database, cache, http)`, and re-raise
2. **Complete `LifecycleManager.shutdown()`**
   Still in `lifecycle_manager.py`, replace the `# YOUR CODE HERE` marker inside `shutdown()`:
   - Return early if `self._initialized` is `False` or `self._shutdown_in_progress` is `True`; otherwise set `self._shutdown_in_progress = True`
   - Inside a `try:`, close clients in **reverse** order — HTTP, cache, then database — wrapping each `close()` in its own `try`/`except` so one failure doesn't block the others (collect error messages into a list)
   - Guard each close with a truthiness check since any client on `self.dependencies` may be `None`
   - In a `finally:` block, reset `self._initialized = False`, `self._shutdown_in_progress = False`, and `self.dependencies = None`, then log whether shutdown was clean or had errors
3. **Run the Lifecycle Manager Directly**
   Verify startup and shutdown fire in the correct order:

```
uv run python lifecycle_manager.py
```

Validation Checkpoint: You see database → cache → HTTP init banners, the health check reports `healthy`, and shutdown banners appear in reverse order.

4. **Run the Agent End-to-End**
   `agent.py` is already wired up with cache-aside `lookup_order` and `get_external_data` tools that use your managed dependencies. Run it:
   ```
   uv run python agent.py
   ```
   Validation Checkpoint: The first order lookup misses the cache and hits the database; the second lookup of the same order is a cache hit. The service shuts down cleanly at the end.

5. **Run the Test Suite**
   The provided `test_lifecycle.py` exercises your `initialize()` and `shutdown()` plus error paths with fake clients:
   ```
   uv run pytest test_lifecycle.py -v
   ```
   Validation Checkpoint: All tests pass, including `test_lifecycle_cleanup_on_error` (monkey-patches `HTTPClient.initialize` to raise) and `test_shutdown_with_errors` (monkey-patches `DatabaseClient.close` to raise).

8.8. Challenge 1: Implement Detailed Health Checks

Open `challenge1_health_checks.py`. The file provides:

- A `HealthCheckResult` dataclass with `healthy`, `message`, `latency_ms`, `timestamp`
- A `HealthChecker` class that holds a `ManagedDependencies` reference
- A fully-implemented `check_http_client()` you can use as a reference pattern for latency timing, success shape, and failure shape

- A fully-implemented `get_overall_status(results)` that maps counts to `"healthy"`, `"degraded"`, or `"unhealthy"`
- A FastAPI `/health` example at the bottom of the module

Three methods and a report builder are marked `# YOUR CODE HERE`.

Goal: Return structured, time-stamped health information for each dependency with measured latency, then aggregate into a report suitable for a monitoring endpoint.

Your Tasks:

- **`check_database()`** — Time a trivial query against `self.dependencies.database`; return a `HealthCheckResult` with measured latency in milliseconds. Use the same shape as `check_http_client()` — successful path and exception path.
- **`check_cache()`** — Time a set/get round trip against `self.dependencies.cache` using a sentinel key and short TTL. Fail the check (healthy=False) if the round-tripped value doesn't match what you wrote.
- **`check_all()`** — Await all three individual checks (database, cache, http) and return a dict keyed by name. Log each result with its latency so operators can see it in the console.
- **`get_health_report()`** — Call `check_all()`, pass the result to `get_overall_status()`, and return a report dict with `status`, ISO `timestamp`, a `checks` dict where each value has serializable fields (bool `healthy`, str `message`, float `latency_ms`, ISO `timestamp`), and a `summary` dict with counts of total/healthy/unhealthy checks.

Hints:

- Use `time.time()` (or `time.perf_counter()`) at the top and bottom of each check to compute latency; round to two decimal places for readability.
- On exceptions, still return a `HealthCheckResult` with `healthy=False` and the exception text in `message` (truncate if long) — never let an individual check raise out of the class.

- `datetime.now().isoformat()` gives you a JSON-serializable timestamp string.

Validation Checkpoint: Run the module directly to see the report:

```
uv run python challenge1_health_checks.py
```

All three dependencies report healthy with low latency, and the top-level `status` is `"healthy"`.

8.9. Challenge 2: Implement Graceful Shutdown

Open `challenge2_graceful_shutdown.py`. The file provides:

- A `ShutdownConfig` dataclass with `grace_period_seconds`, `force_shutdown_seconds`, and `polling_interval_seconds` defaults
- A `GracefulShutdownManager` class that already implements request-counter bookkeeping (`increment_active_requests`, `decrement_active_requests`, `get_active_requests`, `is_shutdown_initiated`) using an `asyncio.Lock`
- A FastAPI example (`create_app_with_graceful_shutdown`) showing how endpoints use the manager

Three methods and one demo function are marked `# YOUR CODE HERE`.

Goal: Cleanly shut down a running service by refusing new requests, waiting up to a grace period for in-flight requests to finish, and then delegating to the underlying `LifecycleManager`.

Your Tasks:

- **`shutdown()`** — The main orchestration method:
  - Return early if `self._shutdown_complete` is already `True`
  - Under `self._lock`, set `self._shutdown_initiated = True` so new requests are rejected
  - If there are active requests, await `self._wait_for_requests()`

    - Delegate final cleanup to `self.lifecycle_manager.shutdown()`
    - Set `self._shutdown_complete = True` and log start/finish banners
- **`_wait_for_requests()`** — Poll the active-request counter until it reaches zero or the grace period expires:
    - Compute a deadline using `asyncio.get_event_loop().time() + self.config.grace_period_seconds`
    - Loop: break on zero active requests; break with a warning if the deadline has passed; otherwise log progress and `await asyncio.sleep(self.config.polling_interval_seconds)` before checking again
- **`handle_request(request_handler)`** — Wrap an async callable so request tracking is automatic:
    - `await self.increment_active_requests()` before running the handler
    - Use a `try` / `finally` so the counter is decremented even on exceptions
    - Return whatever the handler returns
- **`example_graceful_shutdown()`** — A standalone demo that shows the pattern end-to-end:
    - Build a `LifecycleManager`, initialize it, wrap it in a `GracefulShutdownManager` with a small grace period (a few seconds is fine)
    - Kick off three simulated request coroutines of varying durations — some should finish inside the grace period, at least one should exceed it
    - Wait briefly so the requests become "in-flight", then call `shutdown_manager.shutdown()` and observe the grace-period behavior in the logs
    - Clean up any tasks that are still running (cancel them and gather with `return_exceptions=True`)

Hints:

- Acquire `self._lock` only for the brief state transition — do **not** hold it across the whole `_wait_for_requests()` call, or your tracked requests won't be able to decrement.
- Your `handle_request()` is effectively a lightweight alternative to a full async context manager; callers pass a zero-arg async callable.
- Test the timeout path intentionally — make at least one simulated request last longer than `grace_period_seconds` so you can see the "Grace period expired" warning fire.

Validation Checkpoint: Run the module directly:

```
uv run python challenge2_graceful_shutdown.py
```

You should see the shutdown banner appear while requests are active, countdown logs during the grace period, the warning log when the deadline expires, and finally the `LifecycleManager` shutdown banner.

8.10. Challenge 3: Integrate with FastAPI

If time permits, wire the `LifecycleManager` into a real web service.

Goal: Expose the managed dependencies through a FastAPI application whose startup and shutdown hooks run through your `LifecycleManager`.

Your Tasks:

- Define a FastAPI `lifespan` async context manager that constructs a `LifecycleManager`, calls `initialize()` during startup, `yield`s, and calls `shutdown()` during teardown
- Store the `ManagedDependencies` container somewhere endpoints can read it (a module-level variable or `app.state`)

- Build endpoints (e.g. `GET /orders/{id}` and `GET /health`) that call through `deps.database.query(…)`, `deps.cache.get(…)`, and `manager.health_check()`

Hint: `LifecycleManager.lifespan()` is already an `@asynccontextmanager`; you can drive FastAPI's `lifespan` from it directly.

Validation: Start the app with `uv run uvicorn app:app`, hit the endpoints, and confirm that Ctrl-C triggers the shutdown banner you saw in the Core Lab.

8.11. Challenge 4: Expand the Agent Tools

The provided `agent.py` implements a cache-aside `lookup_order` and a thin `get_external_data` wrapper. Extend it:

Your Tasks:

- Add a `create_order(ctx, customer_id, total)` tool that writes through `ctx.deps.database.execute(…)` and then invalidates any cached entries for that customer
- Add a `cache_stats(ctx)` tool that returns `ctx.deps.cache.get_stats()` so the agent can report cache health on demand
- Add at least one new sample query to `main()` that exercises the new tools

Validation: Run `uv run python agent.py` and observe the agent choosing the new tools when appropriate.

8.12. Review

In this lab you learned:

- How to manage dependency lifecycles with proper initialization and cleanup hooks
- How to implement type-safe configuration management using Pydantic Settings with environment variables

- How to build context managers for automatic resource cleanup even during failures
- How to orchestrate startup and shutdown sequences with a LifecycleManager
- How to override dependencies for testing using fakes and mocks in a pytest suite

Implementing Conversational Memory in an AI Agent

LAB 9

- ✓ Implement conversation history to enable multi-turn dialogues
- ✓ Manage conversation context using Pydantic AI's message history
- ✓ Apply sliding window strategies to limit context size
- ✓ Implement summary-based memory for long conversations
- ✓ Design human-in-the-loop patterns for clarification

9.1. Introduction

Single-turn agents are like amnesiacs—they can't remember what was just said. Conversational agents maintain context across turns, enabling natural back-and-forth dialogue where users can ask follow-up questions, provide clarifications, and build on previous exchanges.

Conversation memory presents challenges:

- **Context windows are limited** - Can't send entire conversation every time
- **Older messages become less relevant** - Need strategies to prioritize recent context
- **Costs increase linearly** - More history = more tokens = higher costs
- **Information density varies** - Some exchanges are critical, others aren't

In this lab, you'll build agents that handle multi-turn conversations intelligently, balancing context retention with cost and performance.

9.2. Framework and Tools

Pydantic AI Message History

Built-in conversation management:

- `message_history` parameter for passing previous exchanges
- Automatic handling of user/assistant message roles
- Methods to retrieve and manipulate history

Memory Management Strategies

- **Full History** - Keep everything (simple but expensive)
- **Sliding Window** - Keep only N recent messages
- **Summary-Based** - Summarize older exchanges
- **Hybrid** - Combine strategies based on importance

9.3. The Scenario

You're building an AI travel planning assistant for a travel agency. The assistant needs to:

- Ask clarifying questions about trip preferences
- Remember details from earlier in the conversation (dates, budget, destinations)
- Make recommendations based on accumulated context
- Handle multi-turn negotiations about itinerary options

Without memory, users would have to repeat information constantly. With memory, the agent builds understanding progressively through natural conversation.

9.4. Project Structure

```
implementing-conversational-memory-in-an-ai-agent/
├── .env.example             # Copy to .env and add your API key
├── conversation.py          # ConversationSession (complete
add_turn) & pre-built ConversationBranch
├── memory_strategies.py     # SlidingWindowMemory.apply() to
implement; Summary, Hybrid & Adaptive pre-built
├── travel_agent.py          # Pre-built runners; complete
run_conversation_with_sliding_window
└── pyproject.toml           # Dependencies (UV format)
```

9.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build a travel planning assistant that maintains conversation context across multiple turns and implements a sliding window memory strategy that limits context size as the dialogue grows.

Expected Outcome: A working conversational agent that references earlier turns in ongoing dialogue, applies a sliding window strategy to cap context size, and reports per-session statistics such as turn count and token usage.

9.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/implementing-conversational-memory-in-an-ai-agent
   ```
2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```
   This will create a virtual environment and install all required packages defined in `pyproject.toml`.
3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```
   Edit `.env` and add your OpenAI API key:
   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```
4. **Review the Starter Files**
   Open each provided Python file and read through the scaffolding before writing any code:
   - `conversation.py` — defines `ConversationMetadata`, `ConversationSession`, and `ConversationBranch`. Class signatures, Pydantic models, *`init`* methods, and `list_branches()` are already

implemented. You will complete the remaining method bodies marked with `# YOUR CODE HERE`.

- `memory_strategies.py` — imports Pydantic AI message types, loads the environment, and defines a pre-built `summary_agent` plus the class skeletons `SlidingWindowMemory`, `SummaryMemory`, `HybridMemory`, and `AdaptiveMemory`. You will implement their `apply()` methods (and a few helpers) marked with `# YOUR CODE HERE`.
- `travel_agent.py` — initializes the main `agent`, the `ClarificationCheck` Pydantic model, and the `clarification_agent`. It provides skeletons for seven conversation-runner functions (`run_basic_conversation`, `run_conversation_with_stats`, `run_conversation_with_sliding_window`, `run_conversation_with_summary`, `run_conversation_with_hybrid`, `run_conversation_with_clarification`, `run_conversation_with_branches`) and a `main()` dispatcher. You will complete the `# YOUR CODE HERE` sections inside each runner and in `main()`.

9.7. Core Lab: Track Turns and Apply a Sliding Window

Most of the scaffolding is pre-built: the main `agent`, the `summary_agent`, the `ConversationSession.get_stats` helper, the `SummaryMemory` / `HybridMemory` / `AdaptiveMemory` classes, the `ConversationBranch` methods, and every conversation runner except one are ready to use. You will wire the two missing pieces that form the core memory pattern.

1. **Record Each Turn in `ConversationSession`**
   Open `conversation.py` and complete `ConversationSession.add_turn(self, result)` at the marked block:
   - Replace `self.history` with the full message list produced by the agent run (Pydantic AI `RunResult` exposes this via `all_messages()`).
   - Increment `self.metadata.turn_count`.

- If `result.usage()` returns a value (it can be `None`), add its `total_tokens` to `self.metadata.total_tokens`.

2. **Implement `SlidingWindowMemory.apply`**
   Open `memory_strategies.py` and complete the marked block inside `SlidingWindowMemory.apply(history)`:
   - If `len(history) ≤ self.window_size`, return `history` unchanged.
   - Otherwise, preserve any messages that contain a system prompt, concatenate them with the last `self.window_size` messages, and return that list. **Important:** Pydantic AI's `ModelMessage` does not expose a top-level `.role` attribute — system prompts live inside `msg.parts` as `SystemPromptPart` instances. Detect them with `any(isinstance(p, SystemPromptPart) for p in getattr(msg, "parts", []))` rather than checking `msg.role`.
3. **Wire Up `run_conversation_with_sliding_window`**
   Open `travel_agent.py` and complete the two marked blocks inside `run_conversation_with_sliding_window`:
   - At setup, create a `ConversationSession` (use `str(uuid.uuid4())` for the session ID) and a `SlidingWindowMemory(window_size=6)`.
   - Inside the loop, call `memory.apply(session.history)` to get the windowed history, pass it to `agent.run_sync` via `message_history`, print the reply, call `session.add_turn(result)`, and print a status line such as `[Memory: <windowed>/<total> messages in context]`.
4. **Run and Verify**
   Launch the program and choose option 3 (sliding window):

   ```
   uv run travel_agent.py
   ```

   Validation Checkpoint: Have a conversation of 8-10 turns. The status line should show the context capped at the window size while the storage count keeps growing. Try options 1 and 2 to confirm `add_turn` also powers the basic and stats runners. After exiting with `quit`, the stats line should show a non-zero token total.

9.8. Challenge 1: Swap In Summary-Based Memory

The pre-built `SummaryMemory` class condenses older exchanges into a single system-prompt summary and keeps the most recent turns verbatim. Study it and exercise it end-to-end.

Goal: Understand how summary-based memory differs from a plain sliding window, and confirm the caching behavior.

Your Task:

- Read through `SummaryMemory.apply` and `SummaryMemory._format_messages` in `memory_strategies.py`. Note how the cached `self.summary` prevents re-summarizing on every turn.
- Launch the program and choose option 4 (summary-based memory). Hold a conversation of at least 10 turns covering multiple trip details (dates, budget, destinations, travelers).
- After summarization kicks in, ask the agent a question whose answer depends on an early detail. Confirm the agent still answers correctly using the summary.
- Extensions (optional):
  - Lower `summarize_after` and `keep_recent` in `run_conversation_with_summary` and rerun. How do the thresholds change the agent's recall?
  - Call `memory.reset_summary()` after a few turns and observe that the next call re-runs the summary agent.

Validation: The runner should print `[Using summary + N recent messages]` once the threshold is crossed, and the agent should still reference earlier-turn details such as dates or budget.

9.9. Challenge 2: Exercise the Hybrid and Adaptive Strategies

The pre-built `HybridMemory` routes between three phases (full history, sliding window, summary + recent) based on conversation length. `AdaptiveMemory` picks among the three underlying strategies automatically.

Goal: Observe all three hybrid phases in a single conversation and compare against `AdaptiveMemory`.

Your Task:

- Run option 5 (hybrid memory) and carry on a long enough conversation to see the printed strategy move from `full history` to `sliding window` to `summary + recent`.
- In `travel_agent.py`, add a small runner that uses `AdaptiveMemory` and wire it into the menu. Compare its behavior against `HybridMemory` on the same conversation shape.
- Extensions (optional):
  - Tune `max_messages`, `summarize_threshold`, and `keep_recent` to control when each phase activates.
  - Log which phase fired on each turn and summarize the distribution after you exit.

Validation: The status line should display all three phases at different points during a conversation of 20+ turns.

9.10. Challenge 3: Exercise Human-in-the-Loop Clarification

Detect ambiguous requests and surface a clarifying question before spending tokens on the main agent.

Goal: Study the pre-built clarification flow and extend it so the agent handles vague queries such as "I want to go somewhere warm" more robustly.

Hint: `ClarificationCheck` (with `needs_clarification: bool`, `question: Optional[str]`, and `confidence: float`), the pre-configured `clarification_agent`, and the runner `run_conversation_with_clarification` are all complete in `travel_agent.py`. Read them before extending.

Your Task:

- Launch the program and choose option 6. Enter the prompt "I want to go somewhere warm" and confirm the runner asks a clarifying question before producing an itinerary.
- Extend `run_conversation_with_clarification`:
  - Loop clarification up to a max number of rounds per turn, not just once.
  - Refuse to call the main agent when `needs_clarification` is true but `confidence` is low — explain that you did not understand and re-prompt.
  - Track the number of clarification rounds per turn and surface it in `get_stats()`.

Validation: With the extensions in place, an initially vague prompt should be able to trigger multiple clarification rounds before the main agent runs, and the stats output should report total clarifications.

9.11. Challenge 4: Exercise Conversation Branching

Let users fork the current conversation, explore a "what-if" thread, and return to the main one.

Goal: Study the pre-built branching code and extend it with additional commands.

Hint: `ConversationBranch` (all four methods) and `run_conversation_with_branches` are complete. The function already handles `/branch`, `/switch`, `/branches`, and `/main`.

Your Task:

- Launch the program and choose option 7. Run `/branch alt`, continue the conversation, then `/switch main` and confirm the two threads have diverged.
- Add new branch commands:
  - `/delete <name>` — remove a branch (protect `"main"`).
  - `/diff <name>` — print the last message on a branch next to the last on `"main"`.
  - Show branch creation time and per-branch turn count under `/branches`.

Validation: `/branch alt` → continue the conversation → `/switch main` — the two threads should have diverged and operations on one branch should never mutate another. New commands should behave as specified.

9.12. Next Steps

Explore Further:

- Implement semantic memory (remember facts across sessions)
- Add conversation persistence (save/load conversations)
- Build conversation analytics dashboard
- Implement forgetting strategies for sensitive information

9.13. Review

In this lab you learned:

- How to maintain conversation history across turns using Pydantic AI's `message_history` parameter
- How to track conversation metadata such as turn count, token usage, and session duration

- How to implement sliding window memory to limit context size and control costs
- How to apply summary-based memory that condenses older exchanges while preserving key facts
- How to design human-in-the-loop clarification patterns for handling ambiguous user input

Interactive Agent with Streaming Output

LAB 10

- ✓ Implement streaming responses for real-time token-by-token output
- ✓ Build interactive agent loops with continuous user engagement
- ✓ Handle structured outputs during streaming
- ✓ Manage conversation state across streaming interactions
- ✓ Implement interrupt and correction mechanisms for streaming agents

10.1. Introduction

Traditional agent responses arrive all at once after seconds of waiting. Streaming responses appear token-by-token as they're generated, dramatically improving perceived responsiveness and user engagement.

Streaming is especially valuable for:

- **Long responses** - Users see progress immediately
- **Interactive applications** - Better conversation flow
- **Real-time feedback** - Users can interrupt if response goes off-track
- **Debugging** - Watch agent reasoning unfold in real-time

However, streaming introduces complexity: managing partial state, handling interruptions, and validating incomplete responses. This lab teaches you to build robust streaming agents.

10.2. Framework and Tools

Pydantic AI Streaming Support

Built-in streaming capabilities:

- `run_stream()` method returns async stream
- Token-by-token iteration with `async for`
- Access to final structured result after completion
- Message history updated automatically

Async/Await Patterns

Streaming requires async programming:

- `async def` for coroutine functions
- `await` for async operations
- `asyncio.run()` to execute async code

10.3. The Scenario

You're building a creative writing assistant that helps authors develop story ideas. The assistant needs to:

- Generate long-form content (character descriptions, plot outlines)
- Provide real-time feedback so writers see ideas develop
- Allow interruptions if generation goes in wrong direction
- Maintain conversation context across multiple story elements

Batch responses would feel laggy and prevent interactive refinement. Streaming enables a natural, collaborative writing experience.

10.4. Project Structure

```
files/
├── .env.example                # Copy to .env and add your API key
├── pyproject.toml              # Dependencies (UV format)
├── writer_agent.py             # Core Lab — basic streaming demo
(provided — read-only)
├── interactive_loop.py         # Core Lab — complete the
interactive session, save helper, and guided flow
├── streaming_character.py      # Core Lab — complete
stream_character() to combine streaming with structured output
├── stream_utils.py             # Streaming utilities (provided —
read-only)
├── structured_streaming.py     # Streaming with Pydantic models
(provided — read-only reference)
├── test_streaming.py           # Pytest suite (provided — read-
only)
├── challenge1_interruptible.py  # Challenge 1 — complete
stream_with_interrupt(), interactive_with_corrections(), threaded
streamer, and main()
└── challenge2_parallel.py      # Challenge 2 — complete
stream_section(), parallel_streaming(), compare(),
generate_framework(), and main()
```

10.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build a creative writing assistant with streaming output that displays responses token-by-token, maintains conversation history across turns, and supports structured output alongside live streaming.

Expected Outcome: A working interactive agent that streams responses in real-time, maintains multi-turn conversation context, displays streaming statistics, and handles structured character profiles while streaming their generation.

10.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/interactive-agent-with-streaming-output
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

4. **Review the Starter Files**
   Your lab directory contains several starter files. Open and read each one so you know what's already available:
   - `writer_agent.py` — A fully working streaming demo. An `agent` is created with a creative-writing system prompt. Three async helpers (`stream_response`, `stream_with_metadata`, and a reference `main`) show the basic `async with agent.run_stream(…)` pattern. Use this file as a reference for the streaming idioms used throughout the lab.
   - `interactive_loop.py` — Core Lab scaffolding. Provides the agent, the `show_history()` printer, a fully wired `main()` with mode selection, the shell of `interactive_session()` (with input handling and the `history` list already initialized), the shell of `save_conversation()`, and the shell

of `guided_story_development()`. Several `# YOUR CODE HERE` markers indicate where your work goes.

- `stream_utils.py` — Provided utilities you'll reference later: `StreamStats`, `StreamingDisplay`, `StreamingProgressBar`, `StreamBuffer`, `StreamLogger`, and convenience functions `stream_with_stats()`, `stream_with_progress()`, and `stream_buffered()`.
- `structured_streaming.py` — Reference implementation showing how to combine streaming with `output_type=…` Pydantic models (`CharacterProfile`, `PlotOutline`, `SceneSetting`). Read-only; use it as a pattern library.
- `streaming_character.py` — Core Lab scaffolding for the structured-streaming task. Imports `CharacterProfile` from `structured_streaming.py`, supplies a `character_agent` configured with `output_type=CharacterProfile`, and provides an empty `stream_character(description)` async function whose body is marked `# YOUR CODE HERE`. The *`main`* block calls `stream_character(…)` with a sample description and prints the returned profile via `CharacterProfile.display()`.
- `test_streaming.py` — Pytest suite exercising basic streaming, history, structured output, the utilities, error handling, performance, and tool integration. Run it to verify your implementation.
- `challenge1_interruptible.py` — Challenge 1 scaffolding. Defines `StreamInterrupt`, the `InterruptibleStreamer` class with `_check_interrupt()` already implemented, the `ThreadedInterruptibleStreamer` class with its input thread, and three demo functions that call methods you'll complete. Several `# YOUR CODE HERE` markers mark your work.
- `challenge2_parallel.py` — Challenge 2 scaffolding. Defines four specialist agents (plot, character, setting, theme), the `ParallelStreamCoordinator` class (with timing fields and `_show_performance_stats()` already implemented), a `sequential_streaming()` reference, the

`StructuredParallelStreaming` class shell, and two demo functions. Your work is in the methods marked `# YOUR CODE HERE`.

10.7. Core Lab: Build an Interactive Streaming Loop

All of your Core Lab work happens inside `interactive_loop.py`. The agent, the `show_history()` helper, and the `main()` mode-selection menu are already supplied; your job is to flesh out the session loop, the save helper, and the guided flow.

1. **Handle Special Commands**
   Fill in the first `# YOUR CODE HERE` block inside `interactive_session()` so the loop reacts to the commands printed in the banner.
   - Normalize `user_input` (case-insensitive compare) and recognize four commands: `quit`, `history`, `clear`, `save`
   - `quit` — print a goodbye line and break out of the loop
   - `history` — call the provided `show_history(history)` and `continue`
   - `clear` — reset the `history` list (and the `turn_count`) and `continue`
   - `save` — call `save_conversation(history, session_start)` and `continue`

   Validation Checkpoint: Starting the session and typing `history` should print `(No messages yet)`. Typing `quit` should exit cleanly.

2. **Stream the Response with History**
   Fill in the second `# YOUR CODE HERE` block so the agent actually answers each turn. This is the core streaming pattern used throughout the lab.
   - Print an "Assistant: " label (keep it on the same line with `end=''` and `flush=True`) so streamed text flows after it
   - Open the stream with `agent.run_stream(…)` as an async context manager, passing `user_input` and the current `message_history=history`

- Iterate the stream with `async for chunk in response.stream_text(delta=True):` and print each chunk with `end=''` and `flush=True` so output renders live
- After the stream closes, print a trailing newline and update `history` from `response.all_messages()` so the next turn sees prior context
- Wrap the stream block in `try` / `except Exception` so a failure logs a short message and lets the loop continue instead of crashing

Hint: Increment `turn_count` **before** awaiting the stream so it stays accurate even if the stream raises.

Validation Checkpoint: Have a multi-turn conversation about a story idea. Each response should stream in real-time. Ask a follow-up question that only makes sense with memory (for example, "What color were the eyes again?") and confirm the agent recalls the earlier turn.

3. **Save Conversation to File**

Fill in the `# YOUR CODE HERE` block inside `save_conversation()` so the `save` command writes the transcript to disk.

- If `history` is empty, print a short `[No conversation to save]` notice and return early
- Generate a filename from `session_start` using `strftime("%Y%m%d_%H%M%S")` (for example, `conversation_20250115_143022.txt`)
- Open the file for writing, then iterate `history` and write each message as `ROLE:\n<content>\n\n` followed by a separator line. Decide the role from `isinstance(msg, ModelRequest)` / `ModelResponse`, and extract content by iterating `msg.parts` and reading `part.content` — the provided `show_history()` uses the same pattern, so mirror it
- Wrap the file-writing block in `try` / `except` so I/O errors print a friendly message instead of crashing the session

Validation Checkpoint: After a few exchanges, type `save`. A `conversation_<timestamp>.txt` file should appear in the current directory with both user and assistant turns.

4. **Guided Story Development**
   Fill in the `# YOUR CODE HERE` block inside `guided_story_development()`. This is a short, linear version of the free-form loop that walks the user through building a story framework step by step — and reinforces the streaming pattern from Step 2.
   - Keep a local `history: list[ModelMessage] = []` that persists across the four prompts so the agent builds on earlier answers
   - Ask four `input()` questions in sequence (suggested: genre, concept choice, protagonist, main conflict) and interpolate each answer into the next agent prompt
   - For each question, print an "Assistant: " label, open `agent.run_stream(…)` with `message_history=history`, stream chunks with `async for … response.stream_text(delta=True)`, and update `history = response.all_messages()` after the stream closes
   - End with a short summary line confirming the framework is complete

   Hint: Reuse the same stream + history-update pattern from Step 2 — do not introduce a new API surface.

   Once all four blocks are filled in, run the script:

   ```
   uv run interactive_loop.py
   ```

   Validation Checkpoint: Both menu options reach their respective flows and the chosen flow streams responses in real time while remembering prior turns. The guided flow's fourth prompt should reference earlier answers (for example, the protagonist name) even though you only pass the new conflict into the prompt.

5. **Stream a Structured Character Profile**
   Open the provided `streaming_character.py` file. It imports `CharacterProfile` from `structured_streaming.py` and supplies a pre-configured `character_agent` (with `output_type=CharacterProfile`); your job is to fill in the `# YOUR CODE HERE` block inside `stream_character(description)`.

The `interactive_loop.py` work above streamed plain text. This step adds the typed-output dimension — combining `agent.run_stream(…)` with `output_type=` and `await response.get_output()` to produce a validated Pydantic model alongside the streamed generation.

Pydantic AI separates the streaming APIs by output kind. `response.stream_text(delta=True)` is for **text-only** responses — agents **without** `output_type=`. Calling it on a structured agent raises `UserError: stream_text() can only be used with text responses`. For typed outputs, use `response.stream_output(debounce_by=…)`, which yields **partial** `CharacterProfile` instances as the model produces enough JSON to parse them. Surfacing those partials gives the user feedback that work is happening, then `await response.get_output()` returns the final validated model.

Your Task inside `stream_character()`:

- Open `character_agent.run_stream(description)` as an async context manager
- Iterate `response.stream_output(debounce_by=0.1)` and print a small progress marker per partial — for example, the partial's `name` field once it appears, or a `.` for each parse update
- After the streaming loop exits, print a trailing newline so the structured output renders cleanly below
- **Inside the same `async with` block,** call `await response.get_output()` to obtain the final validated `CharacterProfile` and `return` it

  Hint: `response.get_output()` on a `StreamedRunResult` is an awaitable method (not an attribute). Call it inside the context manager — once `async with` exits, the streamed result is finalized and `get_output()` is no longer reachable through that response handle. Each `partial` yielded by `stream_output()` is a `CharacterProfile` instance with whatever fields the model has produced so far; required fields may not

be set yet, so use `partial.name` defensively (e.g., guard with `if partial.name:` before printing).

Run the script:

```
uv run streaming_character.py
```

Validation Checkpoint: Progress markers stream to the terminal as fields arrive (the character's name appears once the model has produced enough JSON to parse it), followed by a structured `Character Profile: <name>` block printed by `CharacterProfile.display()`. The final block must include all six schema fields (name, age, occupation, personality_traits, backstory, motivation), confirming the `output_type=CharacterProfile` validation ran cleanly.

10.7.1. Explore the Provided Utilities

With the Core Lab working, skim `stream_utils.py` and `structured_streaming.py` so you know what's available for production use:

- `StreamingDisplay.display_stream(stream)` — character/word/chunk counting with an optional `on_chunk` callback
- `StreamingProgressBar` — animated spinner to show while waiting for the first chunk
- `StreamBuffer` — buffered flushing to reduce flicker on fast streams
- `stream_with_stats(agent, prompt, verbose=…)` — one-shot streaming with metrics reporting
- `structured_streaming.py` — examples of combining `agent.run_stream()` with `output_type=CharacterProfile|PlotOutline|SceneSetting`

You can run either file directly (`uv run stream_utils.py`, `uv run structured_streaming.py`) to see the utilities in action.

10.7.2. Run the Tests

```
uv run pytest test_streaming.py -v
```

Validation Checkpoint: All tests should pass. If any test fails, match your implementation to the behavior described in the failing assertion.

10.8. Challenge 1: Implement Interrupt Mechanism

Open the provided `challenge1_interruptible.py` file. It contains the starter scaffolding:

- Imports (including `select`, `sys`, `threading`) and API-key loading
- A `StreamInterrupt` exception class
- `InterruptibleStreamer` with `__init__()` (stores the agent and detects platform) and `_check_interrupt()` (fully implemented — uses `select.select` on Unix/Mac to read a single character non-blockingly)
- The shell of `InterruptibleStreamer.stream_with_interrupt()` and `InterruptibleStreamer.interactive_with_corrections()` — signatures and docstrings supplied, bodies marked `# YOUR CODE HERE`
- `ThreadedInterruptibleStreamer` with `__init__()` and `_input_thread_func()` already written, and `stream_with_interrupt_threaded()` body marked `# YOUR CODE HERE`
- Three demo functions (`demo_basic_interrupt`, `demo_interactive_corrections`, `demo_conversation_with_interrupts`) that call the methods you'll complete
- An empty `main()` marked `# YOUR CODE HERE`

Goal: Allow users to stop a stream mid-generation, optionally provide feedback, and restart with refinement.

Hint: Inside the `async for chunk` loop, call `self._check_interrupt()` once per chunk and raise `StreamInterrupt` when the returned character matches the interrupt key. Catch the exception in the surrounding `try`/`except` and return

whatever chunks you've already collected along with a `was_interrupted=True` flag. Call `await asyncio.sleep(0)` inside the loop so the event loop can actually pick up the keyboard read.

Your Task in `stream_with_interrupt()`:

- Print a prompt line telling the user which key stops the stream
- Open `self.agent.run_stream(prompt, message_history=message_history or [])` as an async context manager
- Stream chunks, collecting each into a list and printing them live
- Before printing each chunk, poll `_check_interrupt()`; if it returns the configured interrupt key, raise `StreamInterrupt`
- Return a tuple of `(joined_text, updated_history, was_interrupted)` — on success, pull the updated history from `response.all_messages()`; on interrupt, return the original history

Your Task in `interactive_with_corrections()`:

- Loop up to `max_retries + 1` times. Each iteration calls `stream_with_interrupt()` with the current prompt and history
- If the stream completed (not interrupted), return the response
- If interrupted, ask the user for a correction via `input()`; if the user gives no feedback, return the partial response; otherwise, rebuild the prompt to include the initial prompt plus the user's correction and continue the loop
- When retries run out, return whatever the last call produced

Your Task in `stream_with_interrupt_threaded()` (cross-platform fallback):

- Reset `self.interrupted` to `False` and start a daemon thread running `self._input_thread_func`
- Stream chunks inside `async with self.agent.run_stream(…)`; before printing each chunk, check `self.interrupted` and return the partial response when it flips to `True`
- Insert a small `await asyncio.sleep(…)` between chunks so the flag is checked promptly

- Return the same `(joined_text, history, was_interrupted)` tuple shape as the non-threaded version

Your Task in `main()`:

- Detect the platform. On Unix/Mac, show a three-option menu that dispatches to the three demo functions. On Windows, run the threaded demo directly.

Validation: Start a long generation, press the interrupt key (`s` on Unix/Mac, Enter on Windows), supply feedback, and confirm the agent restarts with your correction folded into the prompt. The session should never crash if you don't interrupt.

10.9. Challenge 2: Implement Streaming with Multiple Agents

Open the provided `challenge2_parallel.py` file. It contains the starter scaffolding:

- Imports and four specialist agents: `plot_agent`, `character_agent`, `setting_agent`, `theme_agent` — each with its own system prompt
- `ParallelStreamCoordinator` class with `__init__()` (initializes `results`, `start_times`, `end_times` dicts) and `_show_performance_stats()` (fully implemented — prints per-agent timings and overall parallel throughput)
- The shell of `ParallelStreamCoordinator.stream_section()` and `ParallelStreamCoordinator.parallel_streaming()` — signatures, docstrings, and return types supplied, bodies marked `# YOUR CODE HERE`
- The shell of `compare_parallel_vs_sequential()` — marked `# YOUR CODE HERE`
- A fully implemented `sequential_streaming()` reference you can use for comparison
- `StructuredParallelStreaming` class with an inner `StoryFramework` Pydantic model (and its `save_to_file` method) already supplied; only `generate_framework()` is marked `# YOUR CODE HERE`

- Two demo functions (`demo_basic_parallel`, `demo_structured_parallel`) that call the methods you'll complete
- An empty `main()` marked `# YOUR CODE HERE`

Goal: Coordinate streaming from multiple specialist agents concurrently so a full story framework (plot, characters, setting, themes) appears in roughly the time of the slowest agent rather than the sum of all four.

Hint: `asyncio.gather()` schedules coroutines concurrently. Each coroutine must be self-contained — take the agent and prompt as arguments so you can launch several in parallel from `parallel_streaming()`.

Your Task in `stream_section()`:

- Print a section header using the `title` (wrap in the supplied `color_code` if provided so the four agents are visually distinguishable)
- Record the start time into `self.start_times[title]` (use `asyncio.get_event_loop().time()`)
- Open `agent.run_stream(prompt)` as an async context manager, collect chunks, and print them live
- After the stream closes, record `self.end_times[title]`, store the joined text in `self.results[title]`, and return it

Your Task in `parallel_streaming()`:

- Print a banner identifying the concept
- Build four `stream_section` coroutines — one per specialist agent — and pass a distinct prompt and color code for each section (plot, characters, setting, themes)
- Launch them concurrently with `asyncio.gather(…)` and await the result
- When `show_performance` is `True`, call `self._show_performance_stats()`
- Return a dict keyed by section name (`plot`, `characters`, `setting`, `themes`) mapping to each stream's text

Your Task in `compare_parallel_vs_sequential()`:

- Pick a test concept (any short story idea)
- Time a run of `ParallelStreamCoordinator.parallel_streaming(concept)` using `datetime.now()`
- Time a run of the provided `sequential_streaming(concept)` the same way
- Print the two durations side by side and report whether parallel was faster (and by what factor)

Your Task in `StructuredParallelStreaming.generate_framework()`:

- Delegate to `ParallelStreamCoordinator.parallel_streaming(concept, show_performance=False)` to collect the four section results
- Construct and return a `StoryFramework(…)` populated from the returned dict, with `generated_at=datetime.now()`

Your Task in `main()`:

- Print a short menu for the three demos and dispatch based on user input

Validation: All four agents should start at the same time and finish in roughly the wall-clock time of the slowest one — dramatically faster than the sequential reference. The structured demo should produce a valid `StoryFramework` that saves cleanly to disk.

10.10. Next Steps

Explore Further:

- Implement streaming to web clients using Server-Sent Events (SSE)
- Add streaming animations and visual effects
- Build voice-enabled streaming (speech-to-text input, text-to-speech output)
- Implement partial result caching during streaming

10.11. Review

In this lab you learned:

- How to implement token-by-token streaming responses using `run_stream()` and async iteration
- How to build interactive conversation loops that maintain state across streaming turns
- How to add progress indicators and statistics to monitor streaming performance
- How to handle structured outputs alongside live streaming for combined text and data extraction
- How to implement robust error handling with retries for stream failures

Implementing Output Guardrails and Quality Checks

LAB 11

- ✓ Compose multiple guardrail modules into a single agent pipeline
- ✓ Sequence input validation, agent execution, output validation, and PII redaction
- ✓ Return structured result dictionaries that record which guardrails fired
- ✓ Determine when to retry a failed response versus fail closed immediately, based on the failure category

11.1. Introduction

Production agents need guardrails—automated checks that prevent harmful, inappropriate, or low-quality outputs. Without guardrails, agents can:

- Generate offensive or dangerous content
- Leak sensitive information
- Produce factually incorrect responses
- Violate regulatory requirements

This lab teaches you to:

- Validate inputs before processing
- Check outputs against safety policies
- Automatically correct or reject violations
- Detect PII and other sensitive data
- Build layered defense mechanisms

These techniques are essential for deploying agents responsibly in production environments.

11.2. Guardrail Categories

Input Guardrails

- Topic restrictions (forbidden subjects)
- Jailbreak prevention (prompt injection)
- Rate limiting and abuse detection

Output Guardrails

- Content moderation (profanity, violence, hate speech)
- PII detection and redaction
- Factual accuracy checking

- Quality thresholds (length, completeness)

11.3. The Scenario

You're deploying a customer service agent for a financial services company. Regulatory requirements mandate:

- No disclosure of account details without verification
- Detection and redaction of Social Security numbers
- Rejection of queries about competitors' products
- Professional, safe language only

You need comprehensive guardrails to ensure compliance and safety across all interactions.

11.4. Project Structure

```
files/
├── .env.example             # Copy to .env and add your API key
├── pyproject.toml           # Dependencies (UV format)
├── input_guards.py          # Input validation — fully implemented,
read-only
├── output_guards.py         # Output validation — fully implemented,
read-only
├── pii_detector.py          # PII detection — fully implemented,
read-only
└── agent.py                 # Core Lab — complete
handle_query_with_guardrails()
```

11.5. Problem Statement

Time Allocation: 30-45 minutes

Objective: Wire three pre-built guardrail modules (input validation, output validation, PII detection) into a financial services support agent so that unsafe queries are blocked, policy violations fail closed, and PII is redacted from responses.

Expected Outcome: A working `handle_query_with_guardrails()` function that runs each user query through a four-stage pipeline (input check → agent call → output check → PII redaction) and returns a structured result dictionary noting which guardrails fired.

11.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/implementing-output-guardrails-and-quality-checks
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

   Replace `your_api_key_here` with your actual OpenAI API key.

4. **Review the Starter Files**
   Your lab directory contains four Python modules. The three guardrail utilities are fully implemented — your job is only to wire them together in `agent.py`.
   - `input_guards.py` — **Pre-built.** `InputGuardrails.validate_input(query)` returns a `ValidationResult` with `is_valid` plus an `error_message` and `category` when a restricted topic, jailbreak phrase, length violation, or suspicious pattern is detected.
   - `output_guards.py` — **Pre-built.** `OutputGuardrails.validate_output(response)` returns an `OutputValidationResult` with `is_valid`, an `errors` list, a `policy_violations` list, and a `quality_score`.
   - `pii_detector.py` — **Pre-built.** `PIIDetector.redact_pii(text)` returns a `(redacted_text, pii_found)` tuple where `pii_found` is a dict counting SSNs, phone numbers, emails, credit card numbers, and account numbers detected.
   - `agent.py` — Core Lab file. Imports the three modules above and creates a `support_agent` with a financial-services system prompt. The `handle_query_with_retry()`, `interactive_session()`, and `demo()` functions are pre-built. Your task is to implement `handle_query_with_guardrails()`.

11.7. Core Lab: Wire the Guardrail Pipeline

All of your work happens inside `handle_query_with_guardrails(query)` in `agent.py`. The three utility modules (`input_guards`, `output_guards`, `pii_detector`) are already imported, and `support_agent` is already built. Your job is to call them in the right order and return a consistent result dictionary. Build the function step by step; each step below is one chunk of code you add to the body.

1. **Track which guardrails fire**
   Start the function with an empty `triggered` list. You will append a short

string to this list each time a guardrail blocks or modifies the response so the caller (and the pre-built demo) can see what happened.

2. **Run input validation first**
   Call `InputGuardrails.validate_input(query)`. If `is_valid` is `False`, append `"input_validation"` to `triggered` and return immediately with a failure dict:

```
{
    "success": False,
    "response": None,
    "error": input_result.error_message,
    "guardrails_triggered": triggered,
    "pii_redacted": {},
}
```

3. **Call the agent**
   Inside a `try` / `except` block, `await support_agent.run(query)` and read `.output` off the result. On exception, return a failure dict whose `error` explains that the agent call failed.
4. **Run output validation**
   Call `OutputGuardrails.validate_output(response)` to get an `output_result`. Handle two distinct failure modes:
   - If `output_result.is_valid` is `False`, append `"output_validation"` to `triggered` and return a failure dict citing `output_result.errors`.
   - Otherwise, if `output_result.policy_violations` is non-empty, append `"policy_compliance"` to `triggered` and return a failure dict citing those violations.
5. **Redact PII**
   Call `PIIDetector.redact_pii(response)`, which returns a `(redacted_text, pii_found)` tuple. If `pii_found` is non-empty, append `"pii_redaction"` to `triggered` and replace `response` with `redacted_text`.
6. **Return success**
   Return a success dict with `success=True`, the final `response`, `triggered`, the `pii_found` dict, and `output_result.quality_score`.

7. **Run the demo**
   Execute the pre-built demo, which feeds five test queries through your pipeline (valid queries, a restricted topic, an investment-advice query, and a query containing a fake SSN):

   ```
   uv run agent.py --demo
   ```

   Validation Checkpoint: Valid queries should succeed, the restricted-topic query should be blocked by `input_validation`, the investment-advice query should either be blocked at input or fail `policy_compliance`, and the SSN query should come back with the SSN masked and `pii_redaction` in the `guardrails_triggered` list.

8. **Try the interactive loop (optional)**
   Run `uv run agent.py` (without `--demo`) to chat with the guardrail-protected agent. Type `stats` to see counters for each guardrail, and `quit` to exit.

11.8. Challenge 1: Add Retry with Correction Prompts

The pre-built `handle_query_with_retry()` function loops up to `max_retries + 1` times and rebuilds the query as a correction prompt whenever output validation or policy compliance fails. Read through its implementation and extend it.

Goal: Make the retry loop smarter about how it recovers from failures.

Your Task:

- Add logging so each attempt prints the attempt number, the failure category, and a preview of the correction prompt being sent.
- Introduce a maximum total-latency budget (for example, 10 seconds) so the loop gives up early on slow failures instead of always exhausting `max_retries`.

- Add a distinct correction prompt for `policy_compliance` failures versus `output_validation` failures — the current implementation uses one prompt for both.

Validation: Run the demo, force a policy-compliance failure (for example, by asking "What stock should I buy?"), and confirm the correction prompt references the specific violation and that the loop exits under the latency budget.

11.9. Challenge 2: Confidence-Based Guardrails

Implement a guardrail that triggers based on model confidence rather than regex patterns.

Goal: Reject or flag low-confidence responses for human review.

Your Task:

- Add a step to `handle_query_with_guardrails()` that inspects the agent's response for hedging phrases ("I'm not sure", "possibly", "might be") and computes a simple confidence score.
- When confidence falls below a threshold (for example, 0.6), add `"low_confidence"` to `triggered` and return a failure dict routing the query for human review.
- Extend the demo with a query likely to produce a hedged answer and confirm your guardrail catches it.

Hint: For a more advanced version, expose logprobs from the underlying model and average them — but the hedging-phrase heuristic is enough to demonstrate the pattern.

11.10. Next Steps

Explore Further:

- Integrate third-party content moderation APIs (OpenAI Moderation, Perspective API)
- Build custom classifiers for domain-specific policies
- Implement audit logging for all guardrail triggers
- Create override mechanisms for authorized users

11.11. Review

In this lab you learned:

- How to sequence input validation, agent execution, output validation, and PII redaction in a single function
- How pre-built guardrail utilities expose clean APIs (`validate_input`, `validate_output`, `redact_pii`) that a caller simply composes
- How to fail closed — returning a structured error dict — when a guardrail blocks a request
- How to track which guardrails fired so downstream monitoring can report on agent safety

Evaluation of Agent Performance and Cost Management

LAB 12

- ✓ Design comprehensive test suites for evaluating agent performance
- ✓ Measure quality metrics (accuracy, relevance, consistency)
- ✓ Analyze cost-quality trade-offs across model configurations
- ✓ Implement usage monitoring and budget tracking
- ✓ Create performance dashboards and reports

12.1. Introduction

Production agents require continuous evaluation and active cost controls. A model that performs well in isolated tests may regress when prompts change, when new edge cases arrive, or when a cheaper model is swapped in to reduce spend. Without quantitative evaluation and cost visibility, teams cannot make informed trade-offs between quality and price.

In this lab, you will work from a prepared Python script and complete the `# YOUR CODE HERE` regions to implement evaluation logic, model comparison, and per-request cost tracking for a customer support agent.

12.2. Framework and Tools

Pydantic AI Evaluation Pattern

Combines a test suite with runtime instrumentation:

- A test case model captures question, expected keywords, and category
- A result model captures response, pass/fail, tokens, latency, and cost
- An evaluator runs each case against an agent and aggregates results

Cost Tracking Primitives

- Tokens-per-request reported by the model provider
- A configurable cost-per-1K-tokens lookup per model tier
- A persistent log of per-request usage for later analysis

12.3. The Scenario

You maintain a customer support agent that answers policy and logistics questions for an online retailer. Business stakeholders have asked two questions you cannot currently answer:

- "How often does the agent give a complete, accurate answer?"
- "What would it cost per month if we moved to a cheaper or more expensive model tier?"

You will build an evaluation harness that runs a golden dataset of support questions, measures pass rate and latency, totals token spend, and compares multiple model tiers side by side. You will also add a lightweight cost tracker so you can attribute spend to users and days.

12.4. Project Structure

```
files/
├── .env.example             # Copy to .env and add your API key
├── pyproject.toml           # Dependencies (UV format)
├── README.md                # Lab overview (provided — read-only)
└── main.py                  # Core Lab — complete each # YOUR CODE
HERE block
```

12.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build an evaluation and cost-management workflow for a customer support agent that reports quality and spend across model tiers.

Expected Outcome: A runnable evaluator that outputs pass-rate, latency, token, and cost metrics, plus model-comparison insights and a persisted cost log.

12.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/evaluation-of-agent-performance-and-cost-management
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and set `OPENAI_API_KEY` to your actual OpenAI API key. Leave `AI_MODEL` at its default unless you have a reason to change it.

4. **Review the Starter File**
   Open `main.py` and skim the module. Most of the framework is already in place:
   - `TestCase` and `TestResult` dataclasses
   - A `SupportAgent` class that wraps a Pydantic AI agent with an async `handle_query` method
   - An `AgentEvaluator` with `evaluate`, `compare_models`, and `generate_report` already implemented
   - A `CostTracker` with `get_total_cost` and `get_daily_costs` already implemented
   - A fully wired async `main()` that constructs test cases, runs evaluation, compares models, and exercises the tracker

     Two `# YOUR CODE HERE` blocks remain — these are the regions you will complete.

12.7. Core Lab: Build the Evaluation Pipeline

Work through the tasks below in order, filling in each `# YOUR CODE HERE` block in `main.py` as you go.

1. **Score a Single Test Case**
   Complete `AgentEvaluator._evaluate_case(self, agent, case)` so it runs one question through the agent and returns a populated `TestResult`. This is the core of the evaluation pipeline — it produces every metric the report and model comparison depend on.

   Your code should:

   - Record a start timestamp before invoking the agent
   - Await `agent.handle_query(case.question)` to obtain the response string and metadata dict
   - Compute elapsed latency in milliseconds from the start timestamp
   - Walk `case.expected_keywords` and split them into two lists: keywords that appear in the response (case-insensitive) and keywords that do not
   - Set the pass flag based on whether every expected keyword was found
   - Look up the per-1K-tokens rate for `agent.model` in `self.COST_PER_1K_TOKENS`, defaulting to a small fallback if the model is unknown, and compute `cost_usd` from the token count in metadata
   - Return a `TestResult` populated with the case id, question, response, pass flag, found/missing keyword lists, tokens, latency, cost, and a current ISO timestamp

   Hint: `metadata["tokens"]` is already populated by `SupportAgent.handle_query`. Use `datetime.now()` for both the start time and the timestamp field.

   Validation Checkpoint: A single `TestResult` prints with non-zero latency, the response captured, and a cost value proportional to tokens used.

2. **Log Cost-Tracked Requests**
   Complete `CostTracker.log_request(self, user_id, tokens, cost, metadata=None)` so each call persists a record of the request.

   Your code should:

   - Build an entry dict with fields for the current timestamp (ISO format), `user_id`, `tokens`, `cost_usd`, and `metadata` (using an empty dict when none was supplied)
   - Append the entry to `self.costs` so the in-memory accessors (`get_total_cost`, `get_daily_costs`) see the new record
   - Append the entry as a JSON line to `self.log_file` so the record survives between runs

     Hint: Use `json.dumps(entry)` plus a trailing newline and open the log file in append mode.

     Validation Checkpoint: After several `log_request` calls, `tracker.get_total_cost()` returns the expected sum and the log file contains one JSON object per line.

3. **Run the script**

   ```
   uv run python main.py
   ```

   Validation Checkpoint: The script prints an evaluation summary, a model comparison section, and a cost tracking summary, and creates both `evaluation_report.txt` and `cost_log.json` in the current directory.

4. **Inspect the Output**
   Open `evaluation_report.txt` and `cost_log.json`. Confirm that:
   - The report shows a pass rate, total cost, total tokens, and average latency
   - The model comparison section contains one block per model with distinct cost and pass-rate numbers

- `cost_log.json` contains one JSON object per line, each tagged with a `user_id`

12.8. Challenge 1: Extend the Evaluation Loop and Model Comparison

The starter file already implements `AgentEvaluator.evaluate()` and `AgentEvaluator.compare_models()`. Review them, then extend the pipeline:

Your Task:

- Add a fourth test case in a new category (for example: order status, account access) with its own expected keywords
- Add a third model tier to the `compare_models` call in `main()` — for instance, `gpt-4-turbo` — and re-run the script
- Before running, predict which model will have the highest pass rate and which will be cheapest per request, then compare your prediction to the report output

Validation: The console output shows three model blocks in the comparison section, and the fourth test case appears in the detailed results.

12.9. Challenge 2: Implement LLM-as-Judge Evaluation

Keyword matching is a weak proxy for quality — a response can contain every expected keyword and still be unhelpful, or omit a keyword while being entirely correct. Add a second-pass evaluator that asks an LLM to score each response on helpfulness, correctness, and completeness.

Goal: Supplement the keyword-based pass/fail signal with a qualitative score from a judge agent.

Hint: Create a second Pydantic AI agent with a judge-style system prompt and a structured output type (for example, a Pydantic model with three numeric score

fields and a short rationale). Call the judge on each `(question, response, expected)` triple after the baseline evaluation runs, and attach the judge output to each `TestResult` or a parallel data structure.

Your Task:

- Define a schema for the judge's output (scores for helpfulness, correctness, completeness, plus a rationale string)
- Build a judge agent that returns that schema when given the question, the agent's response, and any expected response or keywords for context
- Wire the judge pass into the evaluation flow so each case gets a judge score alongside its keyword result
- Print a combined summary that shows both the keyword pass rate and the average judge scores

Validation: The final report distinguishes cases where keywords pass but the judge is unhappy, and vice versa.

12.10. Challenge 3: Set Up Cost Alerting

Add budget guardrails that trigger alerts when daily or per-user spend crosses configurable thresholds.

Goal: Extend `CostTracker` so it can warn callers (or refuse requests) when spend crosses a limit.

Hint: Build the new checks on top of the existing `get_daily_costs()` and `get_total_cost(user_id)` accessors so you are not duplicating aggregation logic.

Your Task:

- Accept daily and per-user budget thresholds when constructing the tracker (or via a setter)
- Add a method that inspects the current day's spend against the daily threshold and returns an alert state (for example: `ok`, `warning`, `over_budget`)

- Add a method that inspects a user's total spend against the per-user threshold and returns a similar state
- Exercise the new methods from `main` with intentionally tight thresholds so you can see each alert state in the output

Validation: Running the script with low thresholds prints alerts; raising the thresholds makes the alerts disappear without changing the underlying evaluation results.

12.11. Next Steps

Explore Further:

- Expand the golden dataset with adversarial and edge-case questions
- Add a regression check that fails CI if the pass rate drops below a threshold
- Stream cost log entries into a real analytics backend (SQLite, BigQuery, etc.)
- Combine the judge scores with keyword results into a single composite quality metric

12.12. Review Questions

- Which metrics best indicate quality regressions in this lab?
- How should model cost and quality be balanced for production traffic?
- What alert thresholds would you set for early spend anomalies?
- What kinds of failures will keyword-based evaluation miss that an LLM judge would catch?

12.13. Validation Checklist

- `uv run python main.py` runs end-to-end without errors.
- Each test case reports pass/fail, latency, tokens, and estimated cost.
- Model comparison outputs cost-quality trade-off metrics for every model you pass in.

- Cost logs are written to disk and readable as JSON lines.
- Challenge changes do not break baseline reporting.

12.14. Success Criteria

You have completed the lab when you can explain:

- what keyword-based evaluation misses and why a judge complements it
- how to compare model tiers with both quality and cost in the same view
- how budget alerts reduce production risk

Orchestrating a Multi-Step Agent Workflow

LAB 13

- ✓ Design complex workflows by breaking tasks into orchestrated steps
- ✓ Implement sequential execution with context passing between steps
- ✓ Handle errors gracefully with retry and fallback strategies
- ✓ Create conditional branching based on intermediate results

13.1. Introduction

Complex agent systems need explicit orchestration: step ordering, shared state, conditional branching, and retry behavior. A single prompt is rarely enough when a task involves research, summarization, validation, and generation — each phase benefits from its own specialized agent and its own view of the accumulated context.

In this lab, you will complete the single-step execution helper at the heart of a `WorkflowOrchestrator` that runs a sequence of pydantic-ai agents and manages shared state between them. The surrounding orchestration loop — retries, conditional skipping, and required-input validation — is provided for you so you can focus on the data-flow core.

13.2. Framework and Tools

Pydantic AI Agents

Each step in the workflow is a `pydantic_ai.Agent` with its own system prompt. Agents are invoked asynchronously via `await agent.run(prompt)`, which returns a result object whose `.output` attribute contains the generated text.

Workflow Orchestration Patterns

- Sequential execution with a shared `WorkflowContext`
- Per-step retry logic with exponential-style backoff
- Optional `condition` callables that can skip steps
- Required-input validation before a step runs
- Status tracking via a `StepStatus` enum

13.3. The Scenario

You are building a research report generator. The pipeline has three phases: a research agent gathers information on a topic, a summarizer condenses it, and an outline agent structures the summary into a report outline. Each agent's output feeds the next. You also need a second workflow that validates an order, then only processes payment and sends confirmation if validation succeeds.

Both workflows need the same orchestrator. Your job is to implement the orchestration engine that makes this possible.

13.4. Project Structure

```
files/
├── .env.example             # Copy to .env and add your API key
├── pyproject.toml           # Dependencies (UV format)
├── README.md                # Solution notes (provided — read-only)
└── workflow.py              # Core Lab — complete _execute_step()
                             # (execute() is pre-built)
```

13.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Complete the per-step execution helper that makes a multi-step agent workflow possible. The orchestration loop (retries, conditional skipping, required-input validation) is provided; your work is the small function that actually runs a single agent and wires its output back into shared state.

Expected Outcome: A working orchestrator that executes steps in order, passes each step's output to the next through a shared context, and produces a structured results dictionary containing per-step status, outputs, and total duration.

13.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/orchestrating-a-multi-step-agent-workflow
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and set `OPENAI_API_KEY` to your actual OpenAI API key. The `AI_MODEL` value can stay at the default.

4. **Review the Starter File**
   Open `workflow.py` and skim the scaffolding before writing any code. You will see:
   - A `StepStatus` enum with PENDING, RUNNING, COMPLETED, FAILED, SKIPPED values
   - A `WorkflowStep` dataclass holding step configuration and runtime state
   - A `WorkflowContext` dataclass with `set()`, `get()`, and `has()` helpers for shared data
   - A `WorkflowOrchestrator` class with `__init__`, `add_step`, `execute`, and `_get_results` already implemented
   - One `# YOUR CODE HERE` region — inside `_execute_step()`
   - Two fully-written example workflows (`research_workflow_example` and `conditional_workflow_example`) that call into your orchestrator

13.7. Core Lab: Implement Step Execution

The outer `execute()` loop is already written — it tracks status, handles retries, skips steps whose condition returns `False`, and validates required inputs. What it needs from you is a helper that actually runs one step's agent and wires the output into the shared context so the next step can use it.

A few scaffolding details you will rely on: `WorkflowContext.data` is the shared dictionary that flows between steps, `WorkflowStep.prompt_template` is a Python format string that can reference keys in `context.data` (e.g. `"Research the topic: {topic}"`), and the example workflows expect each step's output to be stored under the key `step_<step_name>_result` (e.g. a step named `summarize` writes to `step_summarize_result`).

1. **Locate `_execute_step` in `workflow.py`**
   Find the `# YOUR CODE HERE` comment inside the `_execute_step` method of `WorkflowOrchestrator`. This is the only region you need to complete.
2. **Render the prompt template**
   The step's `prompt_template` contains placeholders like `{topic}` or `{step_research_result}` that need to be filled from the shared context. Use Python's `str.format(**mapping)` to substitute values from `self.context.data`:

   ```
   prompt = step.prompt_template.format(**self.context.data)
   ```

   > ⚠ Warning
   >
   > Every placeholder in `prompt_template` must either appear in `step.required_inputs` or be guaranteed present from a prior step's output (e.g., `step_<earlier_step>_result`) — otherwise `str.format` raises `KeyError` at run time. The outer `execute()` method validates `required_inputs` for you, but it does **not** parse the template to confirm every `{…}` placeholder is covered. When you add a new placeholder in a Challenge extension, add it to `required_inputs` too.

3. **Run the step's agent**
   Each step has a `pydantic_ai.Agent` on `step.agent`. Invoke it asynchronously with the rendered prompt:
   ```
   result = await step.agent.run(prompt)
   ```
4. **Store the output in the shared context**
   Use `self.context.set(…)` to persist the agent's text output under the key `step_<step_name>_result` so downstream steps can reference it. The agent's text is on `result.output`:
   ```
   self.context.set(f"step_{step.name}_result", result.output)
   ```
5. **Return the output**
   Return `result.output` so the calling `execute()` method can record it on `step.result` for the final results dictionary.
6. **Run and Verify**
   Execute the full demonstration:
   ```
   uv run python workflow.py
   ```

   Validation Checkpoint: You should see:

   - A banner for the "Research Report Generation" workflow
   - Three numbered steps (research, summarize, create_outline), each reporting "Completed"
   - A duration line and a JSON dump of the results with `"status": "completed"` and a populated `steps` list
   - A banner for the "Order Processing" workflow with `validate_order` completing, followed by either processed or skipped downstream steps depending on the validation agent's response

13.8. Challenge 1: Rebuild the Orchestration Loop Yourself

The provided `execute()` method handles step iteration, status transitions, retries, conditional skipping, and required-input validation. Delete the provided implementation and rebuild it from scratch to prove you understand each moving part.

Goal: Reproduce the orchestration state machine end to end without looking at the provided version.

Your Task:

- **Seed the context.** If `initial_data` was provided, merge it into `self.context.data`, then record `self.context.start_time` so duration can be computed later.
- **Iterate steps in order.** For each step, print a header showing its position and name.
- **Honor conditions.** If a step has a `condition` callable and it returns `False` for the current context, mark the step `SKIPPED` and continue to the next step without running the agent.
- **Validate required inputs.** Before running the agent, verify every key listed in `step.required_inputs` exists in the context. Raise a `ValueError` describing what is missing.
- **Retry on failure.** Mark the step `RUNNING`, then attempt execution up to `retry_count + 1` times. On each attempt, increment `step.attempts`, call `self._execute_step(step)`, and on success mark the step `COMPLETED`, append its name to `context.steps_completed`, and break the retry loop. On exception, if retries remain, print a notice and `await asyncio.sleep(1)` before the next attempt; otherwise re-raise.
- **Handle terminal failure.** If a step ultimately raises, mark it `FAILED`, capture the message on `step.error`, record `end_time`, and return the results without running further steps.
- **Finalize.** After all steps succeed, record `end_time`, print a completion banner with the total duration, and return `self._get_results()`.

Validation: `research_workflow_example` shows three sequential steps progressing RUNNING → COMPLETED, and `conditional_workflow_example` skips `process_payment` and `send_confirmation` whenever the validation agent's output does not contain `VALID`. The final results dictionary reports accurate per-step status, attempts, and total duration.

13.9. Challenge 2: Dynamic Workflow Generation (Take-Home)

This challenge is intended as a post-class extension — sized at roughly 30–40 minutes on top of Core + Challenge 1, beyond the in-class 45-minute budget.

Generate workflow steps dynamically based on input complexity or domain instead of hard-coding the step list.

Goal: Build a factory that composes a `WorkflowOrchestrator` at runtime — the set of steps depends on the user input or a task classification.

Hint: Before running, ask a small classifier (an agent or a rule-based function) to determine the task category, then select which predefined steps to `add_step` onto a fresh orchestrator.

Your Task:

- Implement a classifier that inspects the initial input and returns a category label.
- Write a factory function that returns a configured `WorkflowOrchestrator` based on that label — different categories should produce different step sequences.
- Demonstrate with at least two distinct inputs that take different paths through the factory.

Validation: The same factory call returns different workflows for different inputs, and each generated workflow runs to completion through the existing `execute()` method.

13.10. Challenge 3: Sub-Workflow Delegation (Take-Home)

This challenge is intended as a post-class extension — open-ended, with no canonical reference solution committed. Allow ~30–60 minutes of independent exploration.

Allow a step to trigger a nested workflow whose outputs feed back into the parent context.

Goal: Let one workflow invoke another as if it were a single step, propagating the child's final context into the parent's.

Hint: A `WorkflowStep` can call into a separate `WorkflowOrchestrator.execute()`. Treat the child workflow as a callable that returns a normalized result.

Your Task:

- Add a step whose handler constructs a sub-workflow (its own `WorkflowOrchestrator` instance with its own steps) and `await`s its `execute()`.
- Merge selected fields from the child's final `context.data` back into the parent's context under a single logical key.
- Handle child-workflow failures: decide whether the parent step fails, retries, or records a partial result.

Validation: The parent workflow reports success only when the nested workflow succeeds, and the parent's final results dictionary contains the child's normalized output alongside the sibling steps.

13.11. Next Steps

Explore Further:

- Add structured logging so each step's prompt, result, and duration are recorded for later inspection.

- Introduce output validators on individual steps so the orchestrator can reject a result and retry even when the agent call itself did not raise.
- Replace the linear step list with a directed graph so steps can declare explicit upstream dependencies instead of relying on list order.
- Wire the orchestrator into a job queue to run workflows as background tasks.

13.12. Review Questions

- Why is explicit step-state tracking important in orchestration?
- How do retries and conditional execution change workflow reliability?
- What are the tradeoffs between naming convention (e.g. `step_<name>_result`) and explicit output wiring for passing data between steps?
- When should a workflow spawn a nested sub-workflow or fan out work in parallel instead?

13.13. Validation Checklist

- `uv run python workflow.py` completes without exceptions.
- Required-input validation raises a clear error when a key is missing.
- Conditional skipping produces `SKIPPED` status and does not invoke the agent.
- Retry behavior is observable when an agent call fails transiently.
- Context is updated with per-step outputs under the `step_<name>_result` convention.
- Final results dictionary reports accurate per-step status, attempts, and total duration.

13.14. Success Criteria

You have completed the lab when you can explain:

- How orchestration differs from single-agent prompting
- How shared context enables data flow between specialized agents
- How state and dependency validation improve reliability
- How parallel or nested workflows help scale complex pipelines

Robust Orchestration with Error Handling and Fallbacks

LAB 14

- ✓ Implement checkpointing to save and resume workflow state
- ✓ Design retry strategies with exponential backoff
- ✓ Build circuit breakers to prevent cascading failures
- ✓ Create fallback execution paths for degraded operation
- ✓ Handle partial failures gracefully in multi-step workflows

14.1. Introduction

Production orchestration requires resilience patterns, not just happy-path logic. A workflow that calls several agents in sequence needs to tolerate transient failures, contain damage when a downstream service misbehaves, degrade to a backup path when the primary fails, and undo side effects from already-completed steps when the whole workflow has to abort.

In this lab, you will complete a robust workflow implementation by filling in the `# YOUR CODE HERE` regions of the provided script. The surrounding scaffolding — circuit breaker, step/orchestrator classes, the example workflow construction — is already in place.

14.2. Framework and Tools

Pydantic AI + asyncio

You'll combine Pydantic AI's async `Agent` API with Python's `asyncio` primitives (`wait_for`, `sleep`) to enforce per-call timeouts, apply exponential backoff between retries, and chain fallback execution without blocking the event loop.

Resilience Patterns

- **Circuit Breakers** — Stop calling a failing dependency after a threshold so errors don't cascade.
- **Retries with Backoff** — Recover from transient failures while avoiding thundering-herd behavior.
- **Fallback Agents** — Degrade gracefully to a cheaper or simpler agent when the primary fails.
- **Compensation / Rollback** — Undo the side effects of completed steps in reverse order when a later step fails.

14.3. The Scenario

You're operating a multi-step data-processing pipeline built from agent calls (extract, transform, summarize). In production, one of those calls will eventually time out, hit a rate limit, or return a malformed result. Crashing the entire pipeline on first error is not acceptable — you need retries, a fallback path, and the ability to roll back anything that already succeeded so the system is left in a consistent state.

14.4. Project Structure

```
files/
├── .env.example             # Copy to .env and add your API key
├── pyproject.toml           # Dependencies (UV format)
├── README.md                # Quick-start notes (provided — read-
only)
└── robust_workflow.py       # Core Lab — complete the marked
sections
```

14.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build an orchestrator that combines retries, circuit breakers, fallback agents, and compensation-based rollback into a single resilient workflow.

Expected Outcome: A working script that executes a three-step agent pipeline, demonstrates retry and fallback behavior when a step fails, and runs compensation actions in reverse order when a fatal failure triggers rollback.

14.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/robust-orchestration-with-error-handling-and-fallbacks
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

   Validation Checkpoint: The `.env` file should now exist alongside `pyproject.toml` with both keys populated.

4. **Review the Starter File**
   Open the provided `robust_workflow.py`. Most of the scaffolding is pre-built so you can focus on the orchestrator:
   - A fully implemented `CircuitBreaker` dataclass with `record_success()`, `record_failure()`, and `can_execute()` methods
   - A fully implemented `RobustWorkflowStep.execute()` that handles timeouts, retries with exponential backoff, circuit-breaker checks, and fallback-agent invocation

- A fully implemented `RobustWorkflowOrchestrator._rollback()` that compensates executed steps in reverse order
- An `example()` coroutine that builds primary and fallback agents, defines two compensation actions, and assembles a three-step workflow (`extract`, `transform`, `summarize`)
- `RobustWorkflowOrchestrator.execute()` is the one method marked `# YOUR CODE HERE` — this is what you'll implement in the Core Lab

5. **Run the baseline**

```
uv run python robust_workflow.py
```

Before you implement the marked regions, expect the script to fail or produce no output — the step and orchestrator methods return nothing.

14.7. Core Lab: Build Resilient Orchestration

Your Core Lab work happens inside the single `# YOUR CODE HERE` region of `RobustWorkflowOrchestrator.execute()` in `robust_workflow.py`. The step-level retry/fallback logic, circuit breaker, rollback routine, and compensation stubs are already implemented — you will focus on the orchestrator glue that drives the pipeline and threads context between steps.

The three pipeline stages chain through a shared context dict: each step's prompt template references placeholders such as `{input_data}`, `{extract_result}`, and `{transform_result}`. Your orchestrator must write each step's output under the key `f"{step.name}_result"` so the next step's template can resolve.

1. **Initialize the Workflow Context**
   At the top of `execute()`, make a local copy of `initial_context` so callers aren't mutated, then seed a `_metadata` entry with `start_time` (use `datetime.now()`) and empty `steps_completed` and `steps_failed` lists. Print a banner identifying the workflow by `self.name` so the operator can follow progress.
2. **Iterate Steps and Capture Results**
   Loop through `self.steps` with `enumerate(…, 1)`. For each step, print a

progress header (e.g., `"Step i/N: name"`), `await step.execute(context)`, and store the returned value under `context[f"{step.name}_result"]`. On success, append the step name to `_metadata["steps_completed"]` and append the step itself to `self.executed_steps` (the rollback path reads this list).

3. **Handle Per-Step Failures with Rollback**
   Wrap each step invocation in a `try/except`. On exception, append the step name to `_metadata["steps_failed"]`, `await self._rollback(context)`, finalize `_metadata` with `end_time` and `status = "failed"`, and return the context immediately without running remaining steps.
4. **Finalize a Successful Run**
   After the loop completes, set `_metadata["end_time"]`, set `_metadata["status"] = "completed"`, print a duration summary, and return the context. As a safety net, wrap the whole loop in an outer `try/except` that calls `await self._rollback(context)` and re-raises on any unexpected error.

   Hint: Keep the per-step `try/except` narrow — the failure branch should short-circuit the loop and return, not continue to the next step.

14.7.1. Run and Verify

```
uv run python robust_workflow.py
```

Validation Checkpoint:

- On a clean run, the banner shows each step transitioning from "Step i/N" to a success marker.
- The returned context contains `extract_result`, `transform_result`, and `summarize_result` keys plus a `_metadata` block with `status: completed` and all three step names in `steps_completed`.
- If you temporarily make a step raise (for example, add `raise RuntimeError("boom")` inside a compensation action's wrapping step), the

rollback section runs the already-completed steps' compensations in reverse order.

14.8. Challenge 1: Rewrite Resilient Step Execution

The pre-built `RobustWorkflowStep.execute()` already handles retries, timeouts, and fallbacks. Rewrite it from scratch to deepen your understanding of the pattern.

> ⓘ Important
>
> A production circuit breaker has **three** states — closed, open, and half-open — not two. Your `can_execute()` rewrite must implement the half-open transition: when `is_open` is `True` **and** `timeout_seconds` has elapsed since the last failure, the breaker resets and allows one trial call through (the half-open probe). On a successful trial call, `record_success()` closes the breaker; on another failure, `record_failure()` re-opens it. A two-state breaker (open forever after the threshold trips) is the bug the Workflow Orchestration module's first-cut implementation exhibited — your rewrite is the corrected version.

Your Task:

- Consult `self.circuit_breaker.can_execute()` before doing any work; when it returns `False`, raise a `RuntimeError` that names the step. (Your `can_execute()` must implement the half-open state described above — see `RobustWorkflowStep.circuit_breaker` for the existing fields.)
- Render the prompt by calling `self.prompt_template.format(**context)`.
- Attempt the primary agent up to `self.max_retries + 1` times. Each attempt must run `self.agent.run(prompt)` under `asyncio.wait_for(…)` using `self.timeout_seconds`.
- On success, call `self.circuit_breaker.record_success()` and return `result.output`.

- Distinguish `asyncio.TimeoutError` from other exceptions in the log output. For non-timeout errors, sleep between retries using exponential backoff (`2 ** attempt`) so delays grow 1s, 2s, 4s, ...
- When retries are exhausted, record the failure on the circuit breaker, then invoke `self.fallback_agent` under the same timeout if configured. If no fallback is configured or the fallback also fails, raise a `RuntimeError` indicating the step failed completely.

Validation: A healthy run completes normally. Pointing the primary agent at a bad model name should produce retry messages followed by the fallback agent being invoked. Tripping the breaker, waiting past `timeout_seconds`, then issuing a successful call should close the breaker again — confirming the half-open path works.

14.9. Challenge 2: Rebuild the Rollback and Compensation Path

The pre-built `_rollback()` and `compensate_step1`/`compensate_step2` stubs demonstrate the happy rollback path. Reimplement them and extend them with richer behavior.

Your Task:

- Rewrite `RobustWorkflowOrchestrator._rollback(self, context)` from scratch: return early if `self.executed_steps` is empty, print a rollback banner, iterate `reversed(self.executed_steps)` awaiting `step.compensate(context)` for each, then clear `self.executed_steps`.
- Replace the two compensation stubs (`compensate_step1`, `compensate_step2`) with distinct, observable cleanup logic — for example, writing a JSON "undo" record to disk, decrementing an in-memory counter, or posting a simulated "reversal" event.
- Force a failure in the third step (e.g., by making the `summarize` step raise unconditionally) and confirm both compensations run in reverse order.

Validation: With a forced third-step failure, the rollback section should call your new `compensate_step2` then `compensate_step1` and leave the simulated side-effect store consistent.

14.10. Challenge 3: Implement Saga Pattern

Refactor the workflow so each transactional stage is expressed as an explicit saga step with a paired compensation handler, rather than relying on the optional `compensation_action` parameter.

Goal: Every registered step must declare both a forward action and a compensating action up front, and the orchestrator must refuse to run a step that hasn't supplied both.

Hint: Introduce a small base class (or `Protocol`) that requires both `async def execute(context)` and `async def compensate(context)` methods. Adapt `RobustWorkflowOrchestrator` to accept that contract instead of the current `RobustWorkflowStep`.

Your Task:

- Define a saga-step contract that exposes both an execute and a compensate coroutine as first-class methods.
- Convert at least two of the existing pipeline stages into concrete saga-step classes that implement that contract.
- Update the orchestrator so registering a step without a compensate method is rejected early (before execution starts).
- Demonstrate the rollback path by forcing a failure in the third stage and confirming each prior saga step's `compensate()` is awaited in reverse order.

Validation: Running the refactored workflow should produce the same success output as the Core Lab, and a forced third-stage failure should trigger the compensation chain for every earlier saga step.

14.11. Challenge 4: Adaptive Retry Strategies

Extend `RobustWorkflowStep.execute()` so retry behavior adapts to the **kind** of error the primary agent raises, rather than treating every failure identically.

Goal: Transient errors retry with backoff; fatal errors (bad configuration, validation, auth) short-circuit straight to the fallback or abort path without wasting the retry budget.

Hint: Classify exceptions into categories before the backoff branch. `asyncio.TimeoutError` and typical network/transport errors are retryable; `ValueError`, authentication errors, and schema validation failures generally are not.

Your Task:

- Introduce an error-classification helper that returns whether a given exception is retryable.
- Skip the remaining retry attempts when a non-retryable error is raised, and move directly to the fallback (or failure) path.
- Record enough information (for example, on the circuit breaker or in step metadata) that you could later tune retry thresholds based on observed success rates per error type.
- Demonstrate both paths: a simulated transient error that retries successfully, and a simulated fatal error that aborts retries immediately.

Validation: A transient failure should produce retry log messages before succeeding; a fatal failure should skip straight to the fallback without retrying.

14.12. Review Questions

- What failures should trigger a fallback versus an immediate abort?
- How do circuit breakers prevent cascading incidents in a multi-step pipeline?
- In what order should compensation actions run, and why?

- When is exponential backoff preferable to a fixed retry delay?

14.13. Validation Checklist

- `uv run python robust_workflow.py` executes without syntax or runtime errors.
- Circuit breaker blocks further calls after its failure threshold is exceeded.
- Retries and timeout handling are visible in the log output.
- Fallback paths execute when the primary agent fails.
- Rollback runs compensation actions in reverse execution order on fatal failure.

14.14. Next Steps

Explore Further:

- Persist `_metadata` and intermediate results to disk so workflows can resume from the last completed step.
- Add structured logging (JSON) so retries, fallbacks, and rollbacks are observable in a log aggregator.
- Emit metrics for circuit-breaker state transitions so operators can alert when a dependency is failing.
- Introduce a dead-letter path for inputs that fail every retry and every fallback.

14.15. Review

In this lab you learned:

- How to enforce per-call timeouts on agent invocations using `asyncio.wait_for`
- How to layer retries with exponential backoff on top of a circuit breaker
- How to fall back to a secondary agent when the primary fails

- How to drive a multi-step workflow while recording metadata for each step
- How to implement compensation-based rollback that undoes completed steps in reverse order

Multi-Agent Task Delegation

LAB 15

- ✓ Design role-based multi-agent systems with clear responsibilities
- ✓ Implement structured handoff protocols between agents
- ✓ Build escalation pathways for complex scenarios
- ✓ Coordinate parallel agent execution with result aggregation
- ✓ Handle inter-agent communication failures gracefully

15.1. Introduction

Real-world agent systems rarely rely on a single generalist model. Instead, they split work across specialists — a researcher, an analyst, a writer, a summarizer — and use a coordinator to route each request to the right expert. This pattern keeps individual system prompts focused, lets you tune cost and quality per capability, and enables both parallel fan-out and sequential pipelines.

In this lab, you'll implement the routing and coordination logic that powers a small team of specialist Pydantic AI agents. The specialist agents themselves are provided; your job is the orchestration layer that decides **who** handles **what** and **in what order**.

15.2. Framework and Tools

Pydantic AI Agents

Each specialist is a `pydantic_ai.Agent` with its own system prompt. The coordinator calls them through the async `agent.run()` method.

Asyncio for Concurrency

- `asyncio.gather()` runs independent agent calls in parallel
- `asyncio.run()` bootstraps the async entry point
- Sequential delegation threads each agent's output into the next prompt via string formatting

Capability-Based Routing

Agents advertise a list of `AgentCapability` values (RESEARCH, WRITING, ANALYSIS, etc.) and a numeric `performance_score` and `cost_multiplier`. The router picks candidates that satisfy a requested capability and selects one based on a quality-vs-cost preference.

15.3. The Scenario

You're the platform engineer for a research-publishing team. The team wants a single programmatic entry point that can:

- Fan out the same research question to multiple topic experts in parallel and collect their findings
- Run a sequential pipeline (research → analysis → writing → summarization) where each step consumes the previous step's output
- Stay extensible so new specialists can be added without changing caller code

Your job is to build the routing and coordination layer that makes these flows possible.

15.4. Project Structure

```
files/
├── .env.example             # Copy to .env and add your API key
├── pyproject.toml           # Dependencies (UV format)
├── multi_agent.py           # Core Lab — implement two functions
marked # YOUR CODE HERE
└── README.md                # Quick-reference notes (provided —
read-only)
```

15.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Build a multi-agent coordinator that routes tasks to specialists by capability, supports both parallel and sequential delegation, and returns structured metadata for every completed task.

Expected Outcome: A working delegation system that runs a parallel research example across three topics and a sequential report-generation pipeline across four stages, printing each specialist's output as it completes.

15.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/multi-agent-task-delegation
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

4. **Review the Starter File**
   Open `multi_agent.py` and read through it before writing anything. Note what's already provided and which blocks are marked `# YOUR CODE HERE`.

15.7. Core Lab: Implement Routing and Sequential Delegation

Most of `multi_agent.py` is already written — data classes, the specialist-agent factory, `delegate_task`, `parallel_delegation`, example workflows, and `main()` are all provided. Your work targets the two functions marked `# YOUR CODE HERE`: the capability-based router, and the sequential pipeline that threads each agent's output into the next prompt.

1. **Open `multi_agent.py` and read it end-to-end**
   Note the `AgentCapability` enum, the `AgentSpec` dataclass, and the four specialists registered inside `create_specialists()`. The two functions you will implement are `TaskRouter.find_agent` and `MultiAgentCoordinator.sequential_delegation`.
2. **Complete `TaskRouter.find_agent`**
   `find_agent(capability, prefer_quality=True)` returns the best registered `AgentSpec` for a requested capability, or `None` if none advertises it.

   Your Task:

   - Build a `candidates` list of specs from `self.agents.values()` whose `capabilities` include `capability`
   - If the list is empty, return `None`
   - If `prefer_quality` is true, return the candidate with the highest `performance_score`; otherwise return the one with the lowest `cost_multiplier`

     Hint: Python's `max()` and `min()` both accept a `key=` keyword argument.

     Validation: `find_agent(AgentCapability.RESEARCH)` returns the `ResearchBot` spec; `find_agent(AgentCapability.CODING)` returns `None`.

3. **Complete `MultiAgentCoordinator.sequential_delegation`**
   `sequential_delegation(tasks, context=None)` runs a list of `(capability,`

`prompt_template)` tuples in order and threads each result into the next prompt. The provided `sequential_report_example` seeds `context` with `{"topic": "renewable energy"}` and uses placeholders like `{topic}` and `{step_1_result}` in each prompt.

Your Task:

- Default `context` to an empty dict when the caller passes `None`
- Iterate through `tasks` with `enumerate(tasks, 1)` so step numbers start at 1
- For each step: render the prompt with `prompt_template.format(**context)`, `await self.delegate_task(…)`, append the result dict to a results list, and store the agent's output text in `context` under the key `step_{i}_result`
- Return the results list

  Hint: The numeric suffix on your context keys must match the placeholders already in `sequential_report_example` (`step_1_result`, `step_2_result`, `step_3_result`).

  Validation: In the sequential demo, the writing step receives the analyst's output, and the summarization step receives the writer's output — not the original topic.

4. **Run the demos**

```
uv run python multi_agent.py
```

Validation Checkpoint: You should see:

- A "Parallel Execution" banner, three `Delegating to ResearchBot…` lines, and three topic summaries
- A "Sequential Execution" banner with four numbered task blocks, each routed to a different specialist, ending in a final summarized report

15.8. Challenge 1: Implement Round-Robin Load Balancing

Extend `TaskRouter` so repeated requests for the same capability spread across all specialists that advertise it, rather than always picking the single top performer.

Goal: Given two or more agents with the same capability, consecutive calls to a new method (for example, `find_agent_balanced`) should cycle through them deterministically.

Your Task:

- Add a second specialist that advertises an existing capability (for example, a second researcher) so round-robin has something to rotate through
- Track the last assignment per capability on the router
- Expose a new selection method that returns the next specialist in the rotation for a given capability
- Write a short test in *`main`* that calls the new method several times for the same capability and prints the agent names returned, demonstrating the rotation

Hint: A `Dict[AgentCapability, int]` that stores the last-used index (modulo the candidate count) is enough to make rotation deterministic.

Validation: Four consecutive calls for the same capability across two equivalent specialists should produce the pattern A, B, A, B.

15.9. Challenge 2: Build a Consensus System (Take-Home)

This challenge is intended as a post-class extension — sized at roughly 25–35 minutes on top of Core + Challenge 1, beyond the in-class 45-minute budget. No reference implementation is committed; explore the design space independently.

Add a consensus strategy where multiple agents answer the same request and a final combiner resolves disagreements.

Goal: Run the same prompt through two or more specialists with overlapping capabilities, then let a summarizer-style agent reconcile the responses into a single consensus answer.

Your Task:

- Add a new coordinator method (for example, `consensus_delegation(capability, prompt, n_voters)`) that delegates the same prompt to multiple distinct specialists with that capability
- Collect the individual responses into a structured list
- Hand the list to a combiner agent (you can reuse `SummaryBot` or add a dedicated one) with a prompt that asks it to merge the answers into one coherent response
- Return both the individual responses and the combiner's consensus output so callers can inspect either

Hint: Start with majority voting for short categorical answers, or a summarizer-style combiner for longer free-form answers.

Validation: For a prompt with a clear factual answer, the consensus output should agree with the majority of voters. For a prompt with subjective framing, the consensus should integrate multiple perspectives rather than picking one.

15.10. Next Steps

Explore Further:

- Add a fallback policy: when `find_agent` returns `None`, route to a generalist agent instead of raising
- Capture per-task latency and token usage in the result dict and aggregate them across a run
- Introduce a circuit breaker that removes a specialist from the registry temporarily after repeated failures
- Swap the in-memory registry for one backed by a config file so specialists can be added without redeploying

15.11. Review

In this lab you learned:

- How to model specialists with capability tags and performance metadata
- How a coordinator can translate a high-level request into a routed agent call
- How `asyncio.gather` turns a list of independent delegations into a concurrent fan-out
- How sequential delegation threads each step's output into the next prompt via string formatting
- How to extend a coordinator with higher-level strategies like round-robin load balancing and consensus

Designing a Manager-Worker Agent System

LAB 16

- ✓ Design manager-worker architectures with clear role separation
- ✓ Implement task decomposition and delegation patterns
- ✓ Build feedback loops for iterative refinement
- ✓ Manage shared vs. isolated context strategies
- ✓ Handle worker failures with retry and reassignment

16.1. Introduction

Complex problems often exceed what a single agent can handle in one call. A manager-worker architecture splits responsibility: a **manager agent** decomposes a complex request into smaller, focused subtasks and a pool of **worker agents** executes those subtasks, then the manager aggregates the results into a final answer.

In this lab you will complete a hierarchical multi-agent system that decomposes a research request, dispatches the subtasks to specialized workers, and synthesizes the responses into a single report.

16.2. Framework and Tools

Pydantic AI Multi-Agent Pattern

This lab uses ordinary `Agent` instances wired together in a hierarchy rather than any framework-level "manager" primitive:

- Each `WorkerAgent` wraps its own Pydantic AI `Agent` with a specialty-specific system prompt
- The `ManagerAgent` wraps its own `Agent` whose job is decomposition and aggregation
- Coordination is plain Python — `async`/`await` and `asyncio` for parallelism

Task Orchestration

- A `Task` dataclass carries description, status, assigned worker, and result
- A `TaskStatus` enum tracks lifecycle (`PENDING`, `IN_PROGRESS`, `COMPLETED`, `FAILED`)
- Workers are load-balanced by `tasks_completed` so the least-busy worker receives the next assignment

16.3. The Scenario

You are building a research assistant that produces multi-section reports. A single prompt to an LLM can produce a report, but quality suffers when one prompt tries to do research, analysis, and writing all at once.

Your solution instead:

- Routes research gathering to a research-focused worker
- Routes data analysis to an analysis-focused worker
- Routes final writing to a writing-focused worker
- Lets a manager agent plan the subtasks and stitch the outputs together

16.4. Project Structure

```
files/
├── .env.example             # Copy to .env and add your API key
├── pyproject.toml           # Dependencies (UV format)
├── README.md                # Quick-start notes (provided — read-
only)
└── manager_worker.py        # Core Lab file
                             #   Provided: TaskStatus, Task,
WorkerAgent,
                             #
ManagerAgent.decompose_task,
                             #
ManagerAgent._aggregate_results,
                             #
ManagerAgent._execute_sequential,
                             #             assign_worker,
register_worker
                             #   You complete: execute_complex_task,
                             #                 _execute_parallel, and
main()
```

16.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Complete a manager-worker pipeline that decomposes a complex research request, delegates each subtask to a specialized worker agent running in parallel, and aggregates the worker outputs into a cohesive final report.

Expected Outcome: A working Python script where a `ManagerAgent` produces 3-5 subtasks from a single complex prompt, three `WorkerAgent` instances execute those subtasks concurrently with load-balanced assignment, and the manager returns a synthesized final answer along with execution metadata (subtask count, execution mode, duration).

16.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/designing-a-manager-worker-agent-system
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

```
OPENAI_API_KEY=your_api_key_here
AI_MODEL=openai:gpt-5.4-mini
```

4. **Review the Starter File**
   Open the provided `manager_worker.py`. Read through it before you start editing so you know what scaffolding is already in place and which regions you must complete.

16.7. Core Lab: Build the Manager-Worker Flow

All of your work happens inside `manager_worker.py`. Decomposition, aggregation, and the sequential execution loop are already provided — you only need to wire the pipeline together, add parallel dispatch, and build the demo in `main()`.

Key scaffolding you call into: `decompose_task(complex_task)` returns a list of subtask strings; `_aggregate_results(original_task, results)` returns the synthesized final string; `_execute_sequential(tasks)` is the reference loop whose per-task result dict shape your parallel version must match; and inside `_execute_parallel(tasks)` the inner `async def execute_one(task)` helper is already provided — you only dispatch it concurrently.

1. **Open** `manager_worker.py` and locate the three `# YOUR CODE HERE` markers inside `execute_complex_task`, `_execute_parallel`, and `main()`.
2. **Complete** `_execute_parallel(tasks)` — replace the `# YOUR CODE HERE` with one line that launches `execute_one(task)` for every task concurrently and returns the collected list. Use an `asyncio` primitive that preserves input order. (Hint: a single `asyncio.gather(…)` call on a list comprehension is enough.)
3. **Complete** `execute_complex_task(complex_task, parallel=True)` so it runs the full decompose → execute → aggregate pipeline:
   - Record a start timestamp (use `datetime.now()`).
   - Call `await self.decompose_task(complex_task)` to get subtask description strings.

- Wrap each description in a `Task` object with a unique `id` like `"task_1"`, `"task_2"`.
- If `parallel` is `True`, await `self._execute_parallel(tasks)`; otherwise await `self._execute_sequential(tasks)`.
- Await `self._aggregate_results(complex_task, results)` for the final answer.
- Compute duration in seconds and return a dict with keys `original_task`, `subtasks` (count), `results`, `aggregated_result`, `duration_seconds`, and `execution_mode` (`"parallel"` or `"sequential"`).

4. **Complete `main()`** so it demonstrates the system end-to-end:
   - Instantiate a `ManagerAgent`.
   - Instantiate three `WorkerAgent` objects — one each for research, analysis, and writing.
   - Register each worker with `manager.register_worker(…)`.
   - Define a `complex_task` string (e.g. a report that combines research, analysis, and recommendations).
   - Call `result = await manager.execute_complex_task(complex_task, parallel=True)`.
   - Print the headline metadata (`original_task`, `subtasks`, `execution_mode`, `duration_seconds`). The trailing code that prints the aggregated answer and worker statistics is already provided.
5. **Run the pipeline:**

```
uv run python manager_worker.py
```

6. **Verify the output.** Confirm that the manager prints 3-5 numbered subtasks, each worker prints its `[name] Executing: …` line, the aggregated answer appears, and the per-worker task-count summary at the end shows every worker completed at least one task.

16.8. Challenge 1: Rewrite Decomposition and Aggregation From Scratch

The provided `decompose_task` and `_aggregate_results` methods do the mechanical work of prompting the manager and parsing output. Replace them with your own implementations to deepen your understanding of the decomposition/aggregation contract.

For `decompose_task`: Build a user prompt that asks the manager agent for 3-5 clear, actionable subtasks. Await `self.agent.run(prompt)`, then split `result.output` on newlines, strip whitespace, skip empty lines, and strip leading numbering/bullet characters (digits, `.`, `)`, `-`, spaces). Return the cleaned list.

Hint: LLM output is not perfectly deterministic — defensive parsing that tolerates `1.`, `1)`, and `-` prefixes is worthwhile.

For `_aggregate_results`: Select only the entries whose `status` is `"completed"` and build a text block that pairs each subtask description with its returned result. Build a prompt that gives the manager the original task plus the compiled results and asks it to produce a comprehensive final answer. Await `self.agent.run(prompt)` and return `result.output`.

Validation: Rerun the pipeline — decomposition should still produce 3-5 numbering-free strings, and the aggregated answer should reference material from the worker outputs rather than restating the original prompt verbatim.

16.9. Challenge 2: Implement Worker Specialization

Extend `ManagerAgent.assign_worker(task)` so it routes each subtask to the worker whose `specialty` best matches the task description, rather than using pure load balancing.

Hint: Simple keyword matching against `worker.specialty` is enough to get started; you can upgrade to a model-assisted classifier by giving the manager a short classification prompt.

Goal: Rerun the full pipeline and confirm that research-flavored subtasks land on the research worker, analysis-flavored subtasks land on the analysis worker, and so on.

Validation: Inspect the printed `[name] Executing: …` lines — the specialty of the executing worker should be appropriate for each subtask.

16.10. Challenge 3: Implement Hierarchical Management

Add a sub-manager pattern where a top-level manager can delegate a branch of work to a secondary `ManagerAgent` that owns its own worker pool.

Hint: A `ManagerAgent` exposes an `async` entry point (`execute_complex_task`) that returns a string via `aggregated_result`. Wrapping a sub-manager in a lightweight object that matches the `WorkerAgent` interface (async `execute_task(task)` returning a string) lets the top-level manager treat it like any other worker.

Goal: Build a two-level hierarchy — one top-level manager, at least one sub-manager with its own workers, plus at least one "leaf" worker — and run a complex task through it.

Validation: The final aggregated output should include content produced by the sub-manager's workers, and worker statistics from both levels should be non-zero.

16.11. Next Steps

Explore Further:

- Add retry-with-reassignment when a worker call raises
- Persist the `Task` queue so the system can resume after a crash
- Stream partial results back to the user as each worker finishes
- Replace keyword-based worker routing with an LLM-powered router

16.12. Review Questions

- What responsibilities should stay with the manager versus workers?
- When is parallel execution a bad fit for delegated tasks?
- How would you add retry and reassignment for worker failures?

16.13. Validation Checklist

- `uv run python manager_worker.py` completes without exceptions.
- Manager creates 3-5 subtasks from a complex task prompt.
- Worker execution works in parallel mode with concurrent wall-clock behavior.
- Aggregated output includes synthesized content from completed subtasks.
- Challenge features are isolated and testable.

16.14. Success Criteria

You have completed the lab when you can explain:

- how manager-worker separation improves scalability
- when to use parallel vs. sequential task execution
- how to route tasks to specialized workers
- how to structure escalation/hierarchy for complex workloads

Testing an AI Agent with Simulated Backends

LAB 17

- ✓ Create deterministic tests for non-deterministic AI agents
- ✓ Implement simulated model backends for controlled testing
- ✓ Write unit tests that validate tool calls and outputs
- ✓ Test edge cases and error conditions systematically
- ✓ Build fixture patterns for repeatable test scenarios

17.1. Introduction

Testing AI agents is challenging—models are non-deterministic, external APIs are unreliable, and costs add up quickly during testing. Production systems need reliable tests that:

- Run fast and cheaply (no real API calls)
- Produce consistent results (deterministic)
- Cover edge cases (errors, timeouts, malformed inputs)
- Validate behavior (tool calls, outputs, guardrails)

This lab teaches production testing strategies:

- **Simulated backends** - Replace real models with stubs
- **Fixture patterns** - Reusable test scenarios
- **Tool call validation** - Verify agent behavior
- **Edge case coverage** - Handle failures gracefully
- **Integration testing** - Test complete workflows

Without proper testing, agents fail unpredictably in production.

17.2. Testing Challenges

Non-Determinism

- Same prompt → different outputs
- Hard to write assertions

Cost

- Testing with real models is expensive
- Hundreds of test runs add up quickly

External Dependencies

- APIs may be down or rate-limited
- Slow integration tests

Edge Cases

- Rare scenarios hard to trigger
- Need to simulate failures

17.3. The Scenario

You've built a customer support agent with tools (`get_order_status`, `create_ticket`). Before deploying, you need comprehensive tests:

- Unit tests for individual tools
- Agent tests with simulated models
- Integration tests for complete workflows
- Edge case tests (timeouts, errors, malformed data)
- Performance tests (response time, token usage)

The tests must run fast, cheaply, and reliably in CI/CD pipelines.

17.4. Project Structure

```
files/
├── .env.example            # Copy to .env and add your API key
├── pyproject.toml          # Dependencies (UV format)
├── agent.py                # Customer support agent under test
(provided — read-only)
├── mocks.py                # Simulated model backends (provided —
read-only)
├── conftest.py             # PyTest configuration (provided — read-
only)
├── fixtures.py             # Reusable PyTest fixtures (provided —
read-only)
├── test_tools.py           # Core Lab — mostly pre-built; complete
one marked test
├── test_agent.py           # Core Lab — mostly pre-built; complete
two marked tests
├── challenge1_solution.py  # Challenge — golden dataset testing
├── challenge2_solution.py  # Challenge — performance testing
└── challenge3_solution.py  # Challenge — workflow integration tests
(provided in full)
```

17.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Learn how to write deterministic tests for an AI agent by filling in a small number of focused tests against pre-built scaffolding (tools, a `FakeModel` backend, and a `create_test_agent()` factory).

Expected Outcome: A passing unit-test suite that validates a tool's happy path and exercises the `FakeModel` backend — both for a simple scripted response and for a simulated tool call — all without any real API calls.

17.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/testing-an-ai-agent-with-simulated-backends
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: You should see messages indicating successful installation of all packages with no errors.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

   Replace `your_api_key_here` with your actual OpenAI API key. (The simulated-backend tests do not actually call OpenAI, but the agent module loads the key at import time.)

4. **Review the Starter Files**
   Your lab directory contains several starter files. Open and read each one so you know what's already available:
   - `agent.py` — The customer support agent under test. Defines an `OrderStatus` Pydantic model, a simulated `ORDERS` database, a `support_agent` configured with a support-rep system prompt, two tools (`get_order_status`, `create_ticket`), and a `handle_query()` helper.

- `mocks.py` — Simulated model infrastructure. Provides a `FakeModel` class (deterministic text responses), a `ToolCallingFakeModel` class (simulated tool calls), a `create_test_agent()` factory, and helpers `create_fake_model()` and `create_tool_calling_fake_model()`.
- `fixtures.py` — Reusable PyTest fixtures (`simple_agent`, `multi_response_agent`, `tool_calling_agent`, `error_agent`, `polite_agent`, `sample_orders`, `sample_tickets`, `test_queries`, `edge_case_queries`).
- `conftest.py` — PyTest configuration that registers `fixtures.py` as a plugin and defines custom markers (`slow`, `integration`, `unit`).
- `test_tools.py`, `test_agent.py` — Core Lab test files. Most tests are pre-built; a small number are marked `# YOUR CODE HERE` for you to complete.
- `challenge1.py`, `challenge2.py`, `challenge3.py` — Challenge scaffolding (golden datasets, performance, and workflow integration).

17.7. Core Lab: Write the Test Suite

Most of the test suite is already written. You will complete three focused tests — one for a tool and two for the agent — and then run the whole suite. The tools you will test live in `agent.py` (`get_order_status` and `create_ticket`), and the fake-model factory `create_test_agent(response=…, tool_calls=…)` from `mocks.py` is what replaces the real LLM during tests. You will see both in context as you read through the pre-built tests in step 1.

1. **Explore `test_tools.py`**
   Open `test_tools.py`. The `make_ctx()` helper and all but one test are already written. Read through the pre-built tests so you can see the pattern: `make_ctx()` gives you a `RunContext`, then you `await` the tool and assert on its return value (or use `pytest.raises` for the error paths).
2. **Complete `test_get_order_status_success` in `test_tools.py`**
   Find the test at the top of the file marked `# YOUR CODE HERE`. The docstring

tells you exactly what to assert. Look up order ID `ORD-123` in `agent.ORDERS` to confirm the expected values, then write three assertions.

Validation Checkpoint: Run `uv run pytest test_tools.py -v`. All tests (including the one you wrote) should pass.

3. **Complete `test_simple_response` in `test_agent.py`**
   Open `test_agent.py`. The first test is a minimal introduction to `create_test_agent`: build an agent whose only scripted response is `"Thank you for contacting support!"`, run it with any query, and assert that `result.output` equals the scripted string. This is the core fake-backend pattern.

   Validation Checkpoint: Run `uv run pytest test_agent.py::test_simple_response -v` and confirm it passes.

4. **Complete `test_tool_call_simulation` in `test_agent.py`**
   This is the key insight of the lab. Instead of scripting text, you script a **tool call** — the fake backend asks the agent to invoke `get_order_status` with `order_id="ORD-123"`. After the real tool runs (against the simulated `ORDERS` database), the fake backend's second turn produces a text reply. Follow the steps in the docstring to script the tool call, run the agent, and assert on the final output.

   Validation Checkpoint: Run `uv run pytest test_agent.py -v`. Every test should pass.

5. **Run the full test suite**
   Run everything together:

   ```
   uv run pytest -v
   ```

   You should see all unit tests pass under the `unit` marker contributed by `conftest.py`.

   Optional filters (useful for CI):

```
uv run pytest -m unit
uv run pytest -m "not slow"
```

17.8. Challenge 1: Implement Golden Dataset Testing

Open the provided `challenge1.py` file. It contains the starter scaffolding:

- Imports (`pytest`, `json`, `Path`, `dataclass`, `List`, `create_test_agent`).
- A `GoldenExample` dataclass with fields `query`, `expected_response`, `category`, `description`.
- A pre-populated `GOLDEN_DATASET` list covering order-status, general-inquiry, returns, ticket-creation, and tracking queries.
- A `GoldenDatasetTester` class stub you must complete.
- Empty pytest test functions marked `# YOUR CODE HERE`.

Goal: Implement regression testing that detects when previously working scenarios start to fail.

Hint: Responses from the fake agent only need to **contain** the expected keyword — a lowercase substring check is enough for fuzzy matching.

Your Task inside `GoldenDatasetTester`:

- `run_tests(agent)` — iterate the dataset, collect per-example results, and return a summary dict with keys `total`, `passed`, `failed`, `pass_rate`, `results`.
- `test_example(agent, example)` — run the query through the agent and return a dict recording `query`, `category`, `description`, `expected`, `actual`, `passed`, and `error`. Treat a case-insensitive substring match as a pass; catch and record any exception.
- `save_results(filepath)` / `load_results(filepath)` — JSON round-trip for the per-example result dicts.

- `compare_with_baseline(baseline_filepath)` — load a saved baseline and return a dict describing `regressions` (previously passing, now failing), `improvements` (previously failing, now passing), plus their counts.

Your Task in the pytest functions:

- One test that exercises only the `order_status` examples.
- One test that runs the whole dataset through `GoldenDatasetTester` and asserts the pass rate clears a reasonable threshold.
- One test that demonstrates regression detection: run v1 (all passing), save results as baseline, run v2 with one response regressed, reload, and assert `regression_count > 0`. Clean up the baseline file at the end.
- One test that demonstrates fuzzy matching on a single example.

Validation: `uv run pytest challenge1_solution.py -v` passes. Running the file directly (`uv run challenge1_solution.py`) prints a per-example pass/fail report.

17.9. Challenge 2: Build Performance Test Suite

Open the provided `challenge2.py` file. It contains the starter scaffolding:

- Imports (`pytest`, `asyncio`, `time`, `dataclass`, `List`, `create_test_agent`, `FakeModel`).
- A `PerformanceMetrics` dataclass with fields `query`, `latency_ms`, `tokens_used`, `success`, `error`.
- A `PerformanceMonitor` class stub configured with `max_latency_ms`, `max_tokens`, `max_cost_per_query`.
- Empty pytest test functions marked `# YOUR CODE HERE`.

Goal: Measure and enforce performance SLAs (latency, token usage, success rate) for the support agent.

Hint: Use `time.time()` before and after `await agent.run(…)`, multiply the delta by 1000 for milliseconds. Token usage is available on the result via

`result.usage().total_tokens` when present — guard against a missing `usage` attribute.

Your Task inside `PerformanceMonitor`:

- `measure_query(agent, query)` — time a single run, record latency and token usage, store and return a `PerformanceMetrics`. On exception, record `success=False` and the error message.
- `check_sla_compliance(metric)` — return a dict with boolean flags `latency_ok`, `tokens_ok`, `success_ok`, `overall_ok`.
- `get_summary()` — aggregate `total_queries`, `successful_queries`, `success_rate`, `avg_latency_ms`, `max_latency_ms`, `min_latency_ms`, `avg_tokens`, `max_tokens`, `total_tokens` across all recorded metrics.
- `get_violations()` — return a list of dicts describing every metric that fails compliance.

Your Task in the pytest functions:

- Single-query latency check against a strict `max_latency_ms`.
- Token usage check built around a `FakeModel` wired into a fresh `Agent` that reuses `support_agent.system_prompt()`.
- Consistency across ten sequential queries — summary success rate should be 1.0 and average latency below the configured threshold.
- Concurrent load of twenty queries launched via `asyncio.gather(…)`.
- SLA compliance reporting — run several queries and assert `get_violations()` returns a list.
- Performance regression — compare a "new version" latency against a baseline latency and fail if it exceeds 1.5×.
- Sustained load (mark `@pytest.mark.slow`) — 100 sequential queries, success rate at least 99%.
- Latency percentiles — run 50 queries, sort latencies, compute P50/P95/P99, and assert P95 is within the threshold.

Validation: `uv run pytest challenge2_solution.py -v` passes. Running the file directly (`uv run challenge2_solution.py`) prints a demo performance report with any SLA violations listed.

17.10. Challenge 3: Extend the Workflow Integration Tests

Open the provided `challenge3.py` file. It contains a complete set of workflow-level integration tests that exercise end-to-end conversation flows, tool-call sequences, and edge cases:

- Multi-turn conversations chained with `message_history=<previous_result>.new_messages()`.
- Tool-call integration — building an agent with `tool_calls=[…]` and asserting on the final output.
- Sequential tool calls across two agents (`get_order_status` then `create_ticket`).
- Concurrency stress — dispatching ten runs via `asyncio.gather(…)` under `@pytest.mark.slow`.
- Edge cases — very long queries, special characters, error recovery, timeout simulation.

Goal: Study the integration-testing patterns, then extend the file with your own scenarios.

Your Task:

- Run `uv run pytest challenge3_solution.py -v` and confirm every test passes.
- Add a new test that scripts a five-turn conversation with chained `message_history` and asserts every turn's output is non-empty.
- Add a new test that uses two successive tool calls inside a single agent run (hint: extend `tool_calls=[…]` to more than one dict and observe how `ToolCallingFakeModel` dispatches the first call).

- Add a sustained-load test marked `@pytest.mark.slow` that runs 50 sequential queries and asserts all of them return the scripted text.

Validation: All your new tests pass when run under the `integration` marker: `uv run pytest -m integration -v`.

17.11. Next Steps

Explore Further:

- Implement property-based testing (Hypothesis)
- Build snapshot testing for output comparison
- Create load testing for production scenarios
- Implement A/B testing framework

17.12. Review

In this lab you learned:

- How to create deterministic tests for non-deterministic AI agents using simulated model backends
- How to write unit tests for agent tools independently using mock `RunContext` objects
- How to build reusable pytest fixtures for consistent and repeatable test scenarios
- How to test edge cases like empty inputs, oversized queries, and concurrent requests
- How to implement integration tests that validate multi-turn conversation workflows without real API calls

Deploying an Agent with FastAPI and Monitoring

LAB 18

- ✓ Deploy AI agents as production RESTful APIs with FastAPI
- ✓ Implement comprehensive error handling and validation
- ✓ Add structured logging for observability and debugging
- ✓ Track usage metrics (tokens, costs, latency)
- ✓ Implement health checks and readiness probes

18.1. Introduction

Deploying agents to production requires more than just running Python scripts. Production systems need:

- **API layer** - RESTful interface for clients
- **Error handling** - Graceful failure modes
- **Logging** - Request tracing and debugging
- **Metrics** - Usage, performance, cost tracking
- **Health checks** - Kubernetes/monitoring readiness
- **Security** - Authentication, rate limiting
- **Containerization** - Portable deployment

This lab builds a production-ready agent service with FastAPI, covering all essential deployment concerns for reliable operation at scale.

18.2. Production Requirements

Reliability

- Error handling without crashes
- Graceful degradation
- Health check endpoints

Observability

- Structured logging
- Distributed tracing
- Metrics collection

Performance

- Async request handling
- Connection pooling

- Response streaming

18.3. The Scenario

You're deploying your customer support agent as a microservice in a Kubernetes cluster. Requirements:

- RESTful API with OpenAPI docs
- Authentication via API keys
- Rate limiting (10 req/min per key)
- Request/response logging
- Token usage tracking
- Health endpoints for K8s probes
- Docker container for deployment

The service must handle production traffic reliably while providing visibility into performance and costs.

18.4. Project Structure

```
files/
├── .env.example              # Copy to .env and add your API key
├── pyproject.toml            # Dependencies (UV format)
├── Dockerfile                # Container build recipe (provided — read-only)
├── docker-compose.yml        # Multi-service orchestration (provided — read-only)
├── app/
│   ├── __init__.py           # Package marker (pre-built)
│   ├── models.py             # Pydantic request/response schemas (pre-built)
│   ├── agent.py              # Agent setup and process_query() helper (pre-built)
│   ├── middleware.py         # API key auth and rate limiting (pre-built)
│   ├── monitoring.py         # MetricsCollector and HealthChecker (pre-built)
│   └── main.py               # FastAPI app — Core Lab: complete health_check() and query_agent()
└── tests/
    └── test_api.py           # Integration tests (provided — read-only)
```

18.5. Problem Statement

Time Allocation: 40-50 minutes

Objective: Deploy a customer support agent as a production FastAPI microservice with API key authentication, rate limiting, structured logging, usage metrics collection, health check endpoints, and a Docker container for deployment.

Expected Outcome: A running FastAPI service at http://localhost:8000 with documented endpoints for `/v1/query`, `/health`, and `/metrics`, a Dockerfile

that builds and runs successfully, and request logs that include latency and token usage per query.

18.6. Getting Started

1. **Set Up Your Environment**
   Navigate to the lab files directory:
   ```
   cd LabFiles/deploying-an-agent-with-fastapi-and-monitoring
   ```

2. **Install Dependencies with UV**
   Sync the project dependencies:
   ```
   uv sync
   ```

   Validation Checkpoint: All packages install successfully, including `fastapi`, `uvicorn`, `pydantic-ai`, and `python-dotenv`.

3. **Configure API Credentials**
   Create a `.env` file from the example:
   ```
   cp .env.example .env
   ```

   Edit `.env` and add your OpenAI API key:

   ```
   OPENAI_API_KEY=your_api_key_here
   AI_MODEL=openai:gpt-5.4-mini
   ```

4. **Review the Starter Files**
   Your lab directory contains a mostly complete FastAPI application. Open and read each file so you know what's already wired up:
   - `app/models.py` — Pydantic models: `AgentRequest`, `AgentResponse`, `ErrorResponse`, `HealthResponse`, `MetricsResponse`. Pre-built.
   - `app/agent.py` — Pydantic-AI `support_agent` plus `process_query()`, `stream_query_response()`, and `validate_agent_health()` helpers. Pre-built.

- `app/middleware.py` — API key authentication (`verify_api_key`) and sliding-window rate limiting (`rate_limit`). Pre-built.
- `app/monitoring.py` — `MetricsCollector` (tracks requests, tokens, latency, errors, cost) and `HealthChecker` (with caching). Pre-built.
- `app/main.py` — FastAPI app with CORS, exception handlers, startup/shutdown hooks, request logging middleware, `/`, `/metrics`, and `/v1/query/stream` already implemented. Two handler bodies are marked `# YOUR CODE HERE`: `/health` and `/v1/query`.
- `Dockerfile` — Multi-stage container build using `uv`.
- `docker-compose.yml` — Service definitions for the API plus optional Redis and Prometheus/Grafana profiles.
- `tests/test_api.py` — pytest integration tests that exercise health, metrics, authentication, rate limiting, and query endpoints.

18.7. Core Lab: Complete the FastAPI Service

The agent, middleware, monitoring, models, streaming endpoint, and request-logging middleware are all pre-built. You will complete the two route handlers in `app/main.py` that wire the agent into the HTTP layer — the heart of deploying an agent as a service. Each handler body is marked `# YOUR CODE HERE`.

1. **Complete the `/health` handler in `main.py`**
   Call `await health_checker.check()` and `health_checker.get_uptime()`, then return a `HealthResponse` with `status` ("healthy" or "unhealthy"), `timestamp=datetime.now()`, `version="1.0.0"`, `model_available` set to the health result, and `uptime_seconds`. On exception, log the failure and raise `HTTPException(status_code=503, detail="Health check failed")`.
2. **Complete the `/v1/query` handler in `main.py`**
   Authentication and rate limiting are already injected via `Depends(…)` in the signature — focus on the body. Capture a start timestamp, call `process_query(query=request.query, session_id=request.session_id, context=request.context)`, read `tokens_used` from the result, compute `latency_ms`, and call

`metrics_collector.record_request(tokens=tokens_used, latency=latency_ms, success=True)`. Return an `AgentResponse` with `request_id=f"req_{uuid.uuid4().hex[:12]}"`, the `response` text, the `session_id`, and a `metadata` dict containing `tokens_used`, rounded `latency_ms`, and the model name.

3. **Handle errors in `/v1/query`**

   Wrap the body in `try`/`except`. On exception, log with `exc_info=True`, call `metrics_collector.record_request(tokens=0, latency=latency_ms, success=False)`, and raise `HTTPException(status_code=500, detail=f"Failed to process query: {str(e)}")`.

4. **Run the service locally**

   Start the API and open `http://localhost:8000/docs` to try the endpoints.

   ```
   uv run uvicorn app.main:app --reload --port 8000
   ```

5. **Exercise the endpoints**

   Confirm the health probe, run a query with a valid bearer token, and check that metrics update.

   ```
   curl http://localhost:8000/health

   curl -X POST http://localhost:8000/v1/query \
     -H "Authorization: Bearer test_key_123" \
     -H "Content-Type: application/json" \
     -d '{"query": "What are your support hours?"}'

   curl -H "Authorization: Bearer test_key_123" http://localhost:8000/metrics
   ```

   Validation Checkpoint: `/health` returns HTTP 200. The query response includes `request_id`, `response`, and a `metadata` block with non-zero `tokens_used` and `latency_ms`. Calling `/metrics` afterward shows `total_requests` incremented.

6. **Run the integration tests**

   Verify authentication, rate limiting, query, health, and metrics behavior.

```
uv run pytest tests/ -v
```

Validation Checkpoint: All test classes pass.

18.7.1. Build and Run the Container (Optional, Time Permitting)

The provided `Dockerfile` uses a multi-stage `uv`-based build and `docker-compose.yml` wires the API service.

```
docker compose build
docker compose up
```

Validation Checkpoint: The container starts, `http://localhost:8000/health` returns HTTP 200, and Docker's health check transitions to `healthy`.

18.8. Challenge 1: Re-implement the Pre-Built Middleware Yourself

The starter code hands you `verify_api_key()` and `rate_limit()` in `app/middleware.py`. To deepen your understanding, comment out the provided implementations and re-implement them from scratch.

Goal (auth): Validate the `Authorization` header and return the raw API key string.

Requirements:

- Reject missing headers with HTTP 401 and an informative detail message
- Reject headers that do not use the `Bearer <token>` scheme with HTTP 401
- Strip the `Bearer ` prefix, then reject keys not present in the `API_KEYS` dictionary with HTTP 401
- Include a `WWW-Authenticate: Bearer` header on every 401 response
- Log invalid attempts at warning level using only a prefix of the key

Goal (rate limit): Enforce a simple sliding-window rate limit per API key using `rate_limit_store`.

Requirements:

- Initialize an empty list for keys seen for the first time
- Drop timestamps older than `window_seconds` from that key's list
- If the list length is at or above `max_requests`, return `False`
- Otherwise, append `time.time()` and return `True`

Validation Checkpoint: Re-run `uv run pytest tests/` — all authentication and rate-limiting tests still pass.

18.9. Challenge 2: Build Out the Monitoring Layer

Re-implement `MetricsCollector.record_request()`, `MetricsCollector.get_metrics()`, and `HealthChecker.check()` from scratch in `app/monitoring.py` (after commenting out the provided versions).

Requirements for `record_request`: Increment `total_requests`, add `tokens` to `total_tokens`, append `latency` to `latencies`, and increment `total_errors` when `success` is `False`.

Requirements for `get_metrics`: Compute average latency, error rate, estimated cost (using `COST_PER_1K_TOKENS`), and uptime from `start_time`. Return a dict with `total_requests`, `total_tokens`, `total_cost_usd`, `avg_latency_ms`, `error_rate`, and `uptime_seconds`.

Requirements for `HealthChecker.check`: Cache results for `cache_duration` seconds. When force=False and the cache is fresh, return the cached value. Otherwise, call `validate_agent_health()`, store the result and timestamp, and return. On exception, cache `False`.

Validation Checkpoint: The `/metrics` endpoint still returns valid data and back-to-back `/health` calls within the cache window do not hammer the model.

18.10. Challenge 3: Add OpenTelemetry Tracing

Implement distributed tracing with OpenTelemetry for request flow visibility.

Goal: Trace requests across services with spans for each operation.

Hints:

- Install `opentelemetry-instrumentation-fastapi`
- Configure a tracer with Jaeger or a similar backend
- Add custom spans for agent operations (for example, wrap the `process_query()` call)

Approach: See the presentation module for OpenTelemetry setup.

18.11. Challenge 4: Implement Token-Level Response Streaming

Upgrade `/v1/query/stream` so chunks are sent as the model generates them, instead of yielding the full response at once.

Goal: Return responses progressively using Server-Sent Events.

Hints:

- Replace the placeholder body of `stream_query_response()` with pydantic-ai's streaming API (for example, `agent.run_stream()`)
- Yield text fragments as they arrive instead of awaiting the full result
- Handle client disconnects gracefully so a cancelled request doesn't leak the underlying task

Approach: See the presentation module for SSE implementation.

18.12. Next Steps

Explore Further:

- Deploy to Kubernetes with Helm charts
- Implement auto-scaling based on load
- Add caching layer (Redis) for responses
- Set up monitoring dashboards (Grafana)

18.13. Review

In this lab you learned:

- How to deploy an AI agent as a production RESTful API using FastAPI with OpenAPI documentation
- How to implement API key authentication and rate limiting middleware for security
- How to add structured request logging with correlation IDs for observability and debugging
- How to track usage metrics including token counts, costs, and latency per request
- How to containerize an agent service with Docker and configure health check endpoints for orchestration platforms

